

Chapter 4: Jobs

Job objects have been around since Windows 2000, being able to manage one or more processes. Most of their capability revolves around limiting the managed processes in some ways. Their usefulness has grown significantly since Windows 8. On Windows 7 and earlier, a process can be a member of a single job only, while in Windows 8 and later, a process can be associated with multiple jobs.

In this chapter:

- Introduction to Jobs
 - Creating Jobs
 - Nested Jobs
 - Querying Job Information
 - Setting Job Limits
 - Job Notifications
 - Silos
-

Introduction to Jobs

Job objects are visible indirectly in *Process Explorer* if a process is under a job. In that case, a *Job* tab appears in the process' properties (this tab is absent if the process is under no job). Another way to glean at the presence of jobs is to enable the *Jobs* color (brown by default) in *Options / Configure Colors...* Figure 4-1 shows *Process Explorer* with the Jobs color visible with all other colors removed.

Process Explorer - Sysinternals: www.sysinternals.com [PAVEL7540\pavel] (Administrator)

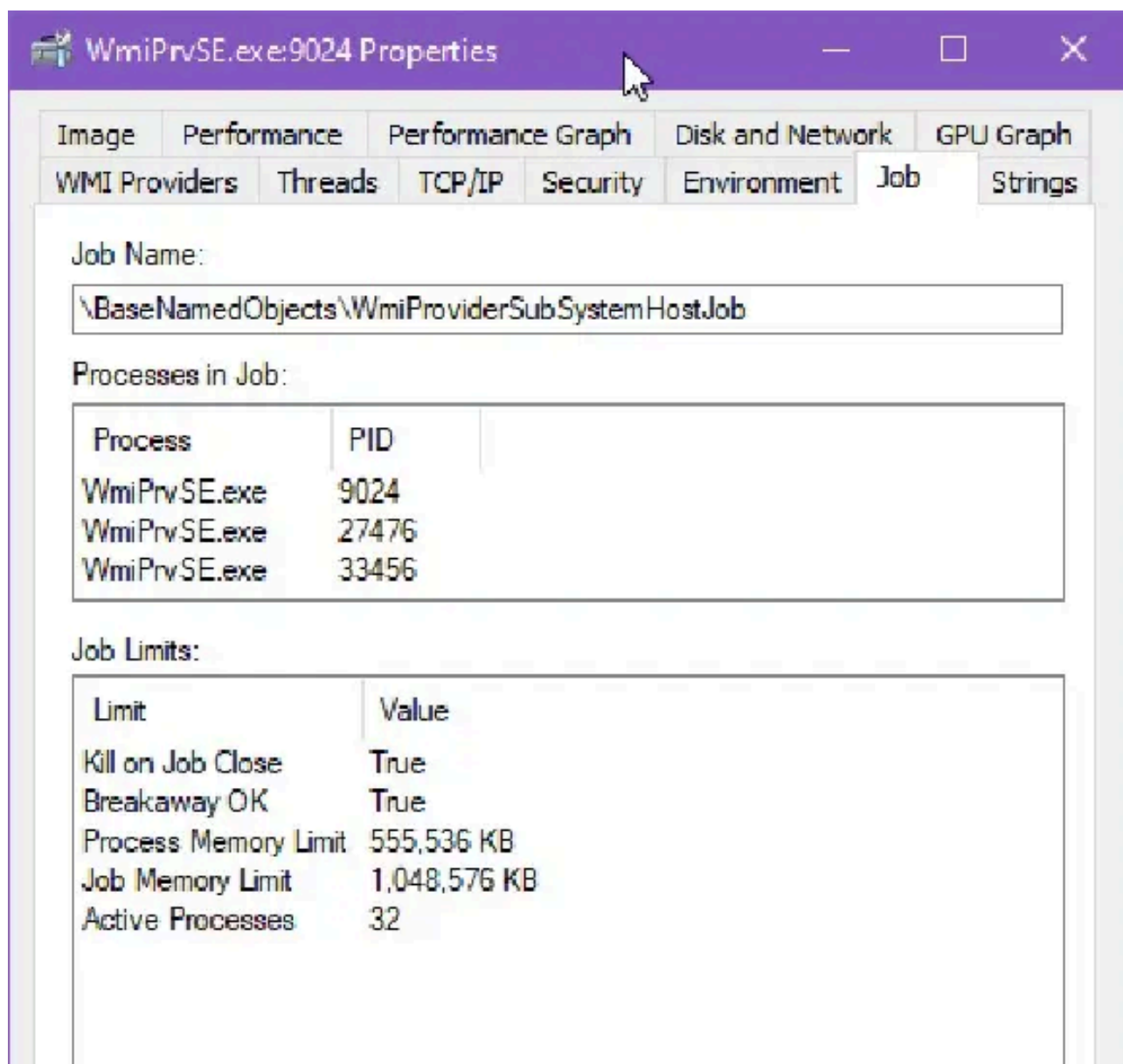
File Options View Process Find Users Help

Process	PID	CPU	Private Bytes	Working Set	Description
Dell.D3.WinSvc.exe	5896	< 0.01	138,440 K	170,208 K	Dell.D3.WinSvc
conhost.exe	5976		6,524 K	10,964 K	Console Window Host
svchost.exe	6052		3,040 K	7,516 K	Host Process for Windows
svchost.exe	6120		1,548 K	7,364 K	Host Process for Windows
svchost.exe	6168		3,984 K	13,232 K	Host Process for Windows
vmms.exe	6236	< 0.01	58,840 K	49,284 K	Virtual Machine Manageme
svchost.exe	6252		5,260 K	15,180 K	Host Process for Windows
svchost.exe	6264		2,316 K	9,704 K	Host Process for Windows
ihl_service.exe	6700		1,260 K	5,984 K	Intel(R) Dynamic Application
IntelCpHeciSvc.exe	6708		1,716 K	7,528 K	IntelCpHeciSvc Executable
sqlservr.exe	6732	0.01	996,680 K	722,020 K	SQL Server Windows NT -
sqlceip.exe	6788		68,060 K	76,928 K	Sql Server Telemetry Client
svchost.exe	6876	< 0.01	3,792 K	13,008 K	Host Process for Windows
firefox.exe	6928	0.55	176,584 K	213,996 K	Firefox Developer Edition
svchost.exe	6940		3,356 K	10,220 K	Host Process for Windows
svchost.exe	6948		1,284 K	5,300 K	Host Process for Windows
lclient.exe	7992		3,104 K	14,004 K	Logi Analytics Client (UNICO
firefox.exe	8076	0.04	700,828 K	654,728 K	Firefox Developer Edition
RtkAudUService64.exe	8304		3,240 K	12,708 K	Realtek HD Audio Universa
nvwmi64.exe	8380	< 0.01	6,704 K	16,468 K	NVIDIA WMI Provider
ServiceHub.RoslynCodeAnalysisService32....	8476	< 0.01	171,348 K	205,376 K	ServiceHub.Host.CLR.x86
RuntimeBroker.exe	8512		4,968 K	29,000 K	Runtime Broker
VBCSCompiler.exe	8584		243,604 K	195,076 K	VBCSCompiler
unsecapp.exe	8696		1,684 K	7,788 K	Sink to receive asynchrono
RtkAudUService64.exe	8844		3,072 K	11,104 K	Realtek HD Audio Universa
conhost.exe	8884		6,516 K	10,956 K	Console Window Host
WavesSvc64.exe	8932		493,904 K	483,396 K	Waves MaxxAudio Service
Docker.Watchguard.exe	8996		536 K	2,584 K	
WmiPrvSE.exe	9024	0.26	48,076 K	59,664 K	WMI Provider Host
slack.exe	9280	< 0.01	198,480 K	215,424 K	Slack
svchost.exe	9316		1,916 K	7,440 K	Host Process for Windows
svchost.exe	9336		1,888 K	7,408 K	Host Process for Windows
conhost.exe	9336		1,888 K	7,408 K	Host Process for Windows

CPU Usage: 18.89% Commit Charge: 37.10% Processes: 370 Physical Usage: 31.09%

Figure 4-1: Brown Processes are in Jobs

If a process is part of a job, its properties show a *Job* tab with details listing the job's name (if any), the processes that are part of the job, and the limits imposed on the job (if any). Figure 4-2 shows a WMI Worker Process (*wmiprvse.exe*) that is part of a named job. Note the job's limits.

Figure 4-2: Job properties in *Process Explorer*

Once a process is associated with a job, it cannot get out. This makes sense, since if a process could be removed from a job, that would make jobs too weak to be useful in many cases.

Creating Jobs

Creating or opening a job is similar to other create/open functions of other kernel object types. Here is the `CreateJobObject` function:

```
HANDLE CreateJobObject(
    _In_opt_ LPSECURITY_ATTRIBUTES pJobAttributes,
    _In_opt_ LPCTSTR                pName);
```

The first argument is the familiar `SECURITY_ATTRIBUTES` pointer, typically set to `NULL`. The second argument is an optional name to set for the new job object. As with other *create*

functions, if a name is provided, and a job with that name exists, then (barring security restrictions), another handle to the existing job is returned. As usual, calling `GetLastError` can reveal whether the job is an existing one, by returning `ERROR_ALREADY_EXISTS`.

Opening an existing job object by name is possible with `OpenJobObject`, defined like so:

```
HANDLE OpenJobObject(
    _In_ DWORD    dwDesiredAccess,
    _In_ BOOL     bInheritHandle,
    _In_ PCTSTR   pName);
```

Most of the arguments should be self-explanatory by now. The first argument specifies the access mask required for the named job object. This access mask is checked against the security descriptor of the job object, returning success only if the security descriptor includes entries that allow the requested permissions. Table 4-1 shows the valid job access masks with a brief description.

Table 4-1: Job Object access masks

Access mask	Description
JOB_OBJECT_QUERY (4)	Query operations against the job, such as <code>QueryInformationJobObject</code>
JOB_OBJECT_ASSIGN_PROCESS (1)	Allows adding processes to the job
JOB_OBJECT_SET_ATTRIBUTES (0x10)	Required to call <code>SetInformationJobObject</code>
JOB_OBJECT_TERMINATE (8)	Required to call <code>TerminateJobObject</code>
JOB_OBJECT_ALL_ACCESS	All possible access to the job

With a job handle in hand, processes can be associated with a job by calling `AssignProcessToJobObject`:

```
BOOL AssignProcessToJobObject(
    _In_ HANDLE hJob,
    _In_ HANDLE hProcess);
```

The job handle must have the `JOB_OBJECT_ASSIGN_PROCESS` access mask, which is always the case when a new job is created, since the caller has full control of the job. The process handle to assign to the job must have the `PROCESS_SET_QUOTA` and `PROCESS_TERMINATE` access mask bits. This means that some processes can never be part of a job, such as protected processes, since this access mask cannot be obtained for such processes.

The following example opens a process given its ID, and adds it into the provided job:

```
bool AddProcessToJob(HANDLE hJob, DWORD pid) {  
    HANDLE hProcess = ::OpenProcess(PROCESS_SET_QUOTA | PROCESS_TERMINATE, FALSE,  
    E, pid);  
    if(!hProcess)  
        return false;  
    BOOL success = ::AssignProcessToJobObject(hJob, hProcess);  
    ::CloseHandle(hProcess);  
    return success ? true : false;  
}
```

Once a process is associated with a job, it cannot break out. If that process creates a child process, the child process is created by default as part of the parent's process job. There are two cases where a child process may be created outside of a job:

- The `CreateProcess` call includes the `CREATE_BREAKAWAY_FROM_JOB` flag and the job allows breaking out of it (by setting the limit flag `JOB_OBJECT_LIMIT_BREAKAWAY_OK` (see the section “Setting Job Limits”, later in this chapter))
- The job has the limit flag `JOB_OBJECT_LIMIT_SILENT_BREAKAWAY_OK`. In this case, any child process is created outside the job without requiring any special flags.

Nested Jobs

Windows 8 introduced the ability to associate a process with more than one job. This makes jobs much more useful than they used to, since if a process you wish to control with a job was already part of a job - there was no way to associate it with another job. A process that is assigned a second job, causes a job hierarchy to be created (if possible). The second job becomes a child of the first job. The basic rules are the following:

- A limit imposed by a parent job affects the job and all child jobs (and hence all processes in those jobs).
- Any limit imposed by a parent job cannot be removed by a child job, but it can be more strict. For example, if a parent job sets a job-wide memory limit of 200 MB, a child job can set (for its processes) a limit of 150 MB, but not 250 MB.

Figure 4-3 shows a hierarchy of jobs created by invoking the following operations (in order):

1. Assign process P1 to J1.
2. Assign process P1 to J2. A hierarchy is formed.
3. Assign process P2 to J2. Process P2 is now affected by jobs J1 and J2.
4. Assign process P3 to J1.

The resulting process/job relationship is depicted in figure 4-3.

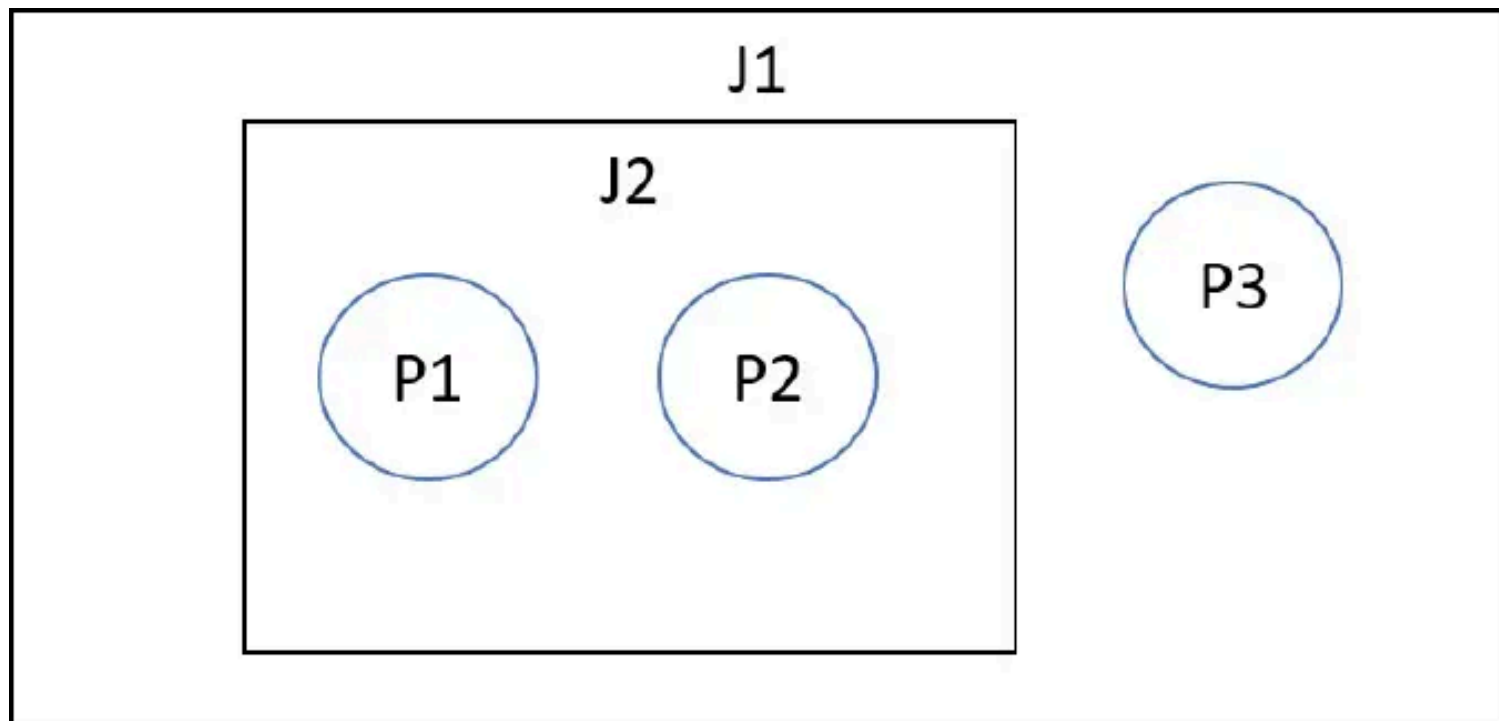


Figure 4-3: Job hierarchy

Viewing job hierarchies is not easy. *Process Explorer*, for example, showing a job's details, includes information for the shown job and all child jobs (if any). For example, viewing the information for job J1 from figure 4-3, three processes would be listed: P1, P2 and P3. Also, since job access is indirect - a *Job* tab is available if a process is under a job - the job shown is the immediate job this process is part of. Any parent jobs are not shown.

The following code creates the hierarchy depicted in figure 4-3.

```

#include <windows.h>
#include <stdio.h>
#include <assert.h>
#include <string>

HANDLE CreateSimpleProcess(PCWSTR name) {
    std::wstring sname(name);
    PROCESS_INFORMATION pi;
    STARTUPINFO si = { sizeof(si) };
    if (!::CreateProcess(nullptr, const_cast<PWSTR>(sname.data()), nullptr, nul\
lptr,
        FALSE, CREATE_BREAKAWAY_FROM_JOB | CREATE_NEW_CONSOLE,
        nullptr, nullptr, &si, &pi))
        return nullptr;

    ::CloseHandle(pi.hThread);
    return pi.hProcess;
}

HANDLE CreateJobHierarchy() {
    auto hJob1 = ::CreateJobObject(nullptr, L"Job1");
    assert(hJob1);
    auto hProcess1 = CreateSimpleProcess(L"mspaint");

    auto success = ::AssignProcessToJobObject(hJob1, hProcess1);
    assert(success);

    auto hJob2 = ::CreateJobObject(nullptr, L"Job2");
    assert(hJob2);
    success = ::AssignProcessToJobObject(hJob2, hProcess1);
    assert(success);

    auto hProcess2 = CreateSimpleProcess(L"mstsc");
    success = ::AssignProcessToJobObject(hJob2, hProcess2);
    assert(success);

    auto hProcess3 = CreateSimpleProcess(L"cmd");
    success = ::AssignProcessToJobObject(hJob1, hProcess3);
    assert(success);
}

```

```
// not bothering to close process and job 2 handles

return hJob1;
}

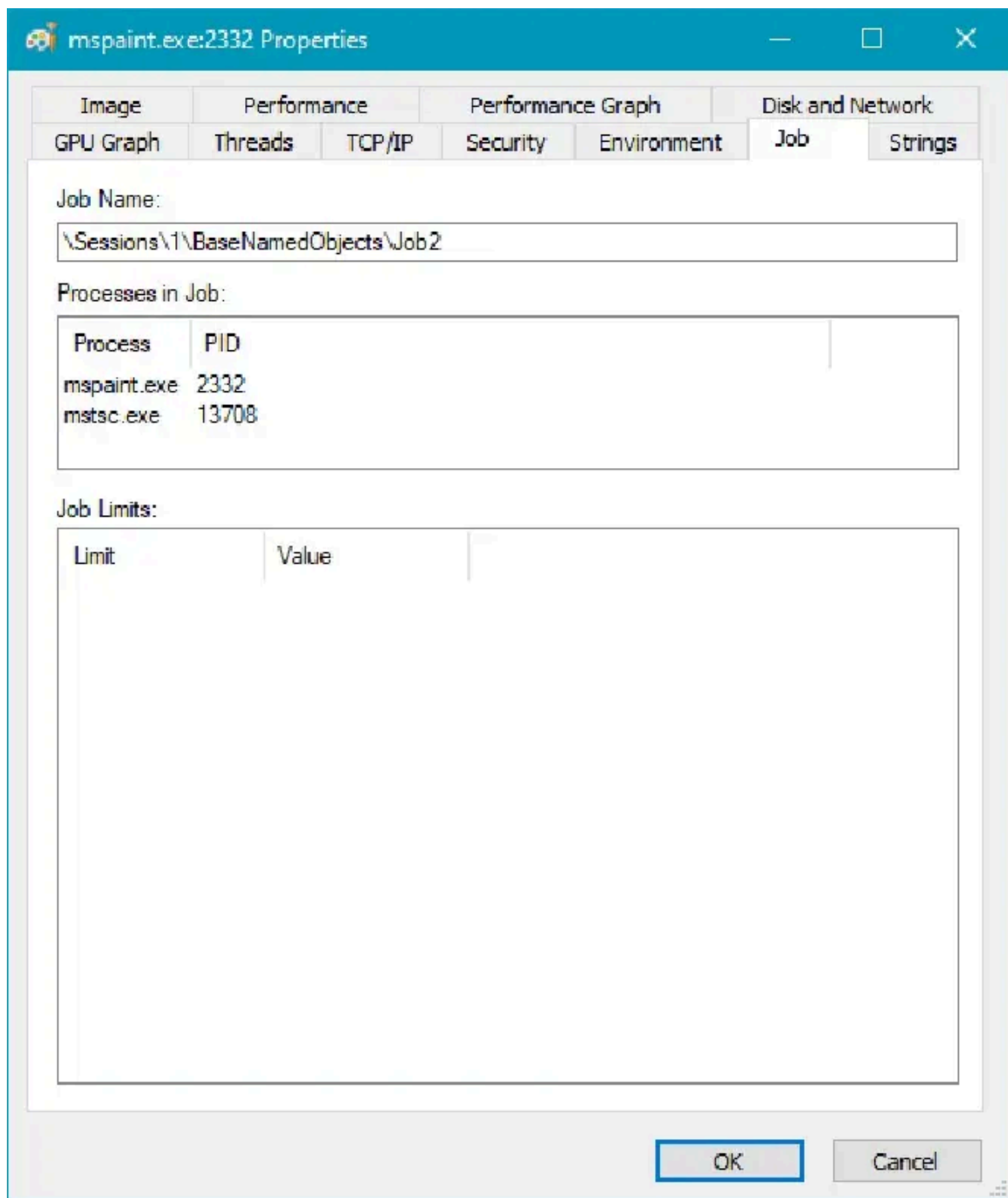
int main() {
    auto hJob = CreateJobHierarchy();
    printf("Press any key to terminate parent job...\n");
    ::getchar();
    ::TerminateJobObject(hJob, 0);
    return 0;
}
```

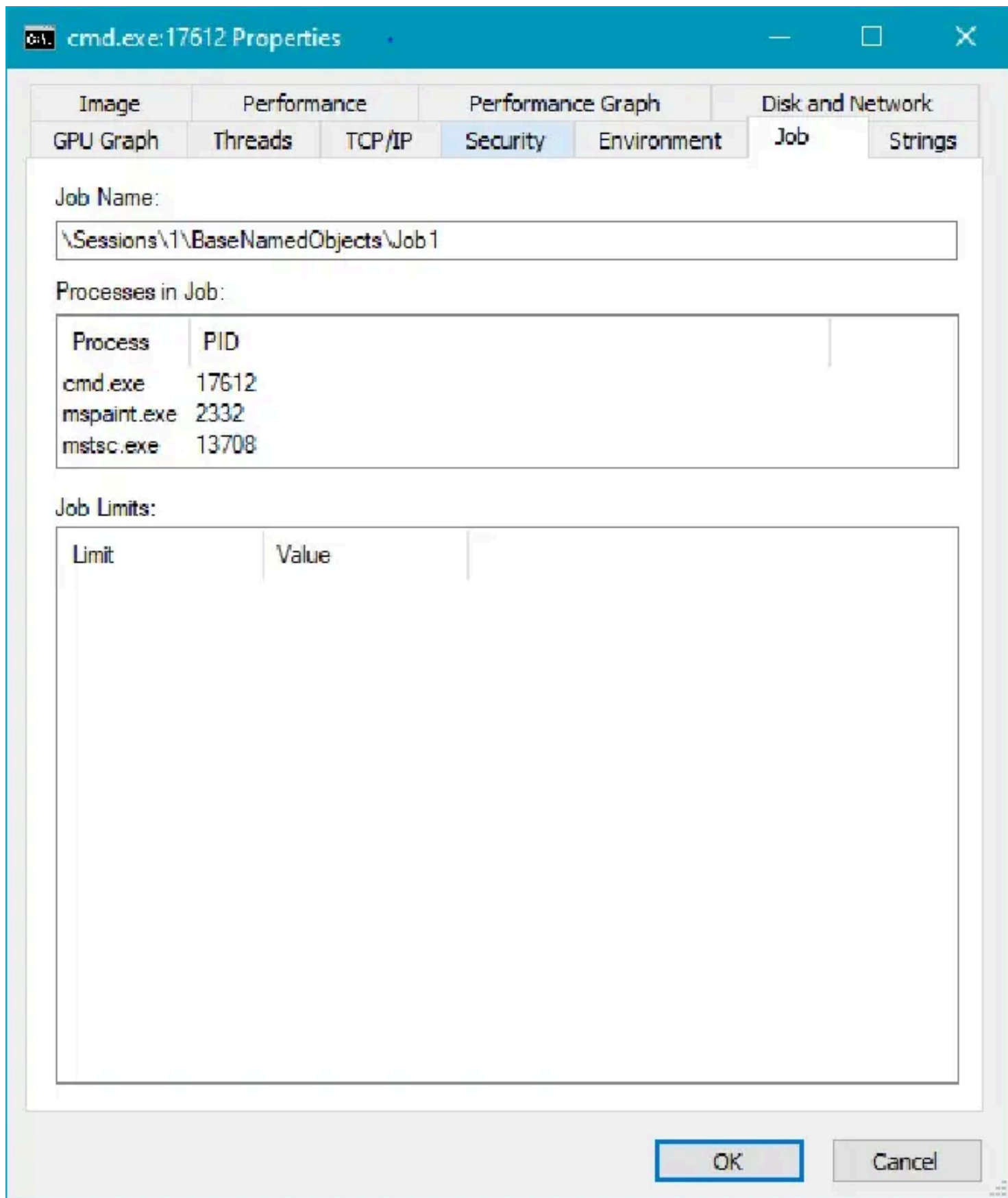
The code is available in the *JobTree* project in this chapter's source code.

The process image names are purposefully different, so they are easier to spot (P1=mspaint, P2=mstsc, P3=cmd). The jobs are named, also for easier identification.

Each process is initially created outside of any job by specifying `CREATE_BREAKAWAY_FROM_JOB` in the `CreateProcess` call. Otherwise, running this application from a process that is already part of a job (such as Visual Studio) would complicate the job hierarchy.

Figure 4-4 shows the job under which *mspaint* is running. Notice it's *Job2*, although *mspaint* is also under *Job1*. Figure 4-5 shows the job under which *cmd* is running, showing the three processes. That's because *cmd* is part of *Job1*, and *Job1* shows all processes including those in child jobs.

Figure 4-4: *Job2* properties in *Process Explorer*

Figure 4-5: *Job1* properties in *Process Explorer*

Viewing job hierarchies is not easy, as there is no documented (or undocumented for that matter) API to enumerate jobs, let alone job hierarchies. A tool I created called *Job Explorer* tries to fill this gap. You can find it in my Github repository at <https://github.com/zodiacon/jobexplorer>.

Running *Job Explorer* while the *JobTree* application is waiting for a key press shows the screenshot in figure 4-6 when the “All Jobs” tree node is selected and the jobs are sorted by name.

Name	Address	Active Process...	CPU Time	Total Processes
\Sessions\1\BaseNamedObjects\WinlogonAccess	0xFFFFD10C24B51080	0	00:00:00.031	1
\Sessions\1\BaseNamedObjects\ProcessJobTracker19496	0xFFFFD10C282207D0	10	00:10:26.718	10
\Sessions\1\BaseNamedObjects\Job2	0xFFFFD10C1B3937D0	2	00:00:02.000	2
\Sessions\1\BaseNamedObjects\Job1	0xFFFFD10C2348A7D0	4	00:00:02.218	4
\BaseNamedObjects\WmiProviderSubSystemSpecialHostJob	0xFFFFD10C16CD0080	0	00:00:00.000	0
\BaseNamedObjects\WmiProviderSubSystemHostJob	0xFFFFD10C16CCF080	0	00:00:54.343	75
	0xFFFFD10C14672060	13	01:48:55.953	32
	0xFFFFD10C1C8D64...	0	00:41:09.281	98

Figure 4-6: *Job Explorer* All Jobs node

Double-clicking on *Job1* expands the job hierarchy on the left and shows the job details on the right, as shown in figure 4-7.

The screenshot shows the *Job Explorer* application window. The left pane displays a tree view of the job hierarchy. The right pane shows the details for the selected job, *Job1*. The details include:

- General:** Address, Name, Active Processes, Total Processes, User Time, Kernel Time, CPU Time, Terminated Processes, Page Faults, Site.
- Processes:** A list of processes running under the job, including *cmd.exe*, *conhost.exe*, *mspaint.exe*, and *notepad.exe*, along with their PIDs and creation times.
- Open Handles:** A list of open handles for the job, including *Handle: 172 (lsAC)* and *JobTree.exe*.

Figure 4-7: *Job Explorer* Job hierarchy view

The job hierarchy is clearly visible in the tree view. Notice that *conhost.exe* process (which is always created when *cmd.exe* is launched) is also part of the same job.

You may be wondering how *Job Explorer* works. I plan to write a blog post about that.

Querying Job Information

A job object keeps track of some basic job statistics even without any special settings. The primary API to query information about a job object is aptly named `QueryInformationJobObject`:

```
BOOL QueryInformationJobObject(
    _In_opt_   HANDLE          hJob,
    _In_       JOBOBJECTINFOCLASS JobObjectInfoClass,
    _Out_      LPVOID          pJobObjectInfo,
    _In_       DWORD           cbJobObjectInfoLength,
    _Out_opt_  LPDWORD          pReturnLength);
```

The `hJob` parameter is the handle to a job - which must have the `JOB_QUERY` access mask; however, as is hinted by the SAL annotation, a `NULL` value is a valid value, pointing to the job the calling process is under (if any). In this way, a process can query information that may be pertinent to its execution, such as any memory limits imposed by the job. If the job is nested, then the immediate job is the one queried. `JOBOBJECTINFOCLASS` is an enumeration of the various pieces of information that can be queried. For each type of information requested, an appropriately-sized buffer must be supplied in the `pJobObjectInfo` argument to be filled by the function. The last argument is an optional value containing the returned data size in the provided buffer, which is useful for some types of queries that return variable-sized data. Finally, just as with most APIs, the function returns a non-`FALSE` value on success.

Table 4-2 summarizes the (documented) information classes available for jobs in query operations.

Table 4-2: `JOBOBJECTINFOCLASS` for documented job query operations

Information class (<code>JobObject*</code>)	Information structure type	Description
BasicAccountingInformation (1)	<code>JOBOBJECT_BASIC_ACCOUNTING_INFORMATION</code>	Basic accounting
BasicLimitInformation (2)	<code>JOBOBJECT_BASIC_LIMIT_INFORMATION</code>	Basic limits
BasicProcessIdList (3)	<code>JOBOBJECT_BASIC_PROCESS_ID_LIST</code>	List of process IDs in the job
BasicUIRestrictions (4)	<code>JOBOBJECT_BASIC_UI_RESTRICTIONS</code>	User interface limits
EndOfJobTimeInformation (6)	<code>JOBOBJECT_END_OF_JOB_TIME_INFORMATION</code>	What happens when the end of job time limit is reached

Table 4-2: JOBINFOCLASS for documented job query operations

Information class (JobObject*)	Information structure type	Description
BasicAndIoAccountingInformation (8)	JOB_OBJECT_BASIC_AND_IO_ACCOUNTING_INFORMATION	Basic and I/O accounting
ExtendedLimitInformation (9)	JOB_OBJECT_EXTENDED_LIMIT_INFORMATION	Extended limits
GroupInformation (11)	USHORT array	(Windows 7+) Processor groups for the job (see chapter 6)
NotificationLimitInformation (12)	JOB_OBJECT_NOTIFICATION_LIMIT_INFORMATION	(Windows 8+) Notification limits
LimitViolationInformation (13)	JOB_OBJECT_LIMIT_VIOLATION_INFORMATION	(Windows 8+) Information on limits violations
GroupInformationEx (14)	GROUP_AFFINITY array	(Windows 8+) Processor group affinity
CpuRateControlInformation (15)	JOB_OBJECT_CPU_RATE_CONTROL_INFORMATION	(Windows 8+) CPU rate limit information
NetRateControlInformation (32)	JOB_OBJECT_NET_RATE_CONTROL_INFORMATION	(Windows 10+) Network rate limit information
NotificationLimitInformation2 (33)	JOB_OBJECT_NOTIFICATION_LIMIT_INFORMATION_2	(Windows 10+) Extended limit information
LimitViolationInformation2 (34)	JOB_OBJECT_LIMIT_VIOLATION_INFORMATION_2	(Windows 10+) Extended limit violation information

Job Accounting Information

As mentioned earlier, a job keeps track of some pieces of information, regardless of any limits imposed on the job. The basic accounting information is available with the `JobObjectBasicAccountingInformation` enumeration and the `JOB_OBJECT_BASIC_ACCOUNTING_INFORMATION` structure defined like so:

```

typedef struct _JOBOBJECT_BASIC_ACCOUNTING_INFORMATION {
    LARGE_INTEGER TotalUserTime;           // total user mode CPU time
    LARGE_INTEGER TotalKernelTime;         // total kernel mode CPU time
    LARGE_INTEGER ThisPeriodTotalUserTime; // same counters as above
    LARGE_INTEGER ThisPeriodTotalKernelTime; // for a "period"
    DWORD TotalPageFaultCount;             // page fault count
    DWORD TotalProcesses;                  // total processes ever existed in the \
    job
    DWORD ActiveProcesses;                 // live processes in the job
    DWORD TotalTerminatedProcesses;        // processes terminated because of limi\
    t violation
} JOBOBJECT_BASIC_ACCOUNTING_INFORMATION, *PJOBOBJECT_BASIC_ACCOUNTING_INFORMAT\
ION;

```

The various times are provided as `LARGE_INTEGER` structures, each holding a 64-bit value in 100 nanosecond units. The “this period” prefix reports times since the recent per-job user/kernel time limits that have been set (if any). These values are zeroed when the job is created and when a new per-job time limit is set.

The following code snippet shows how to make a query call on a job object for basic accounting information:

```

// assume hJob is a job handle
JOBOBJECT_BASIC_ACCOUNTING_INFORMATION info;
BOOL success = QueryInformationJobObject(hJob, JobObjectBasicAccountingInformat\
ion, &info, sizeof(info), nullptr);

```

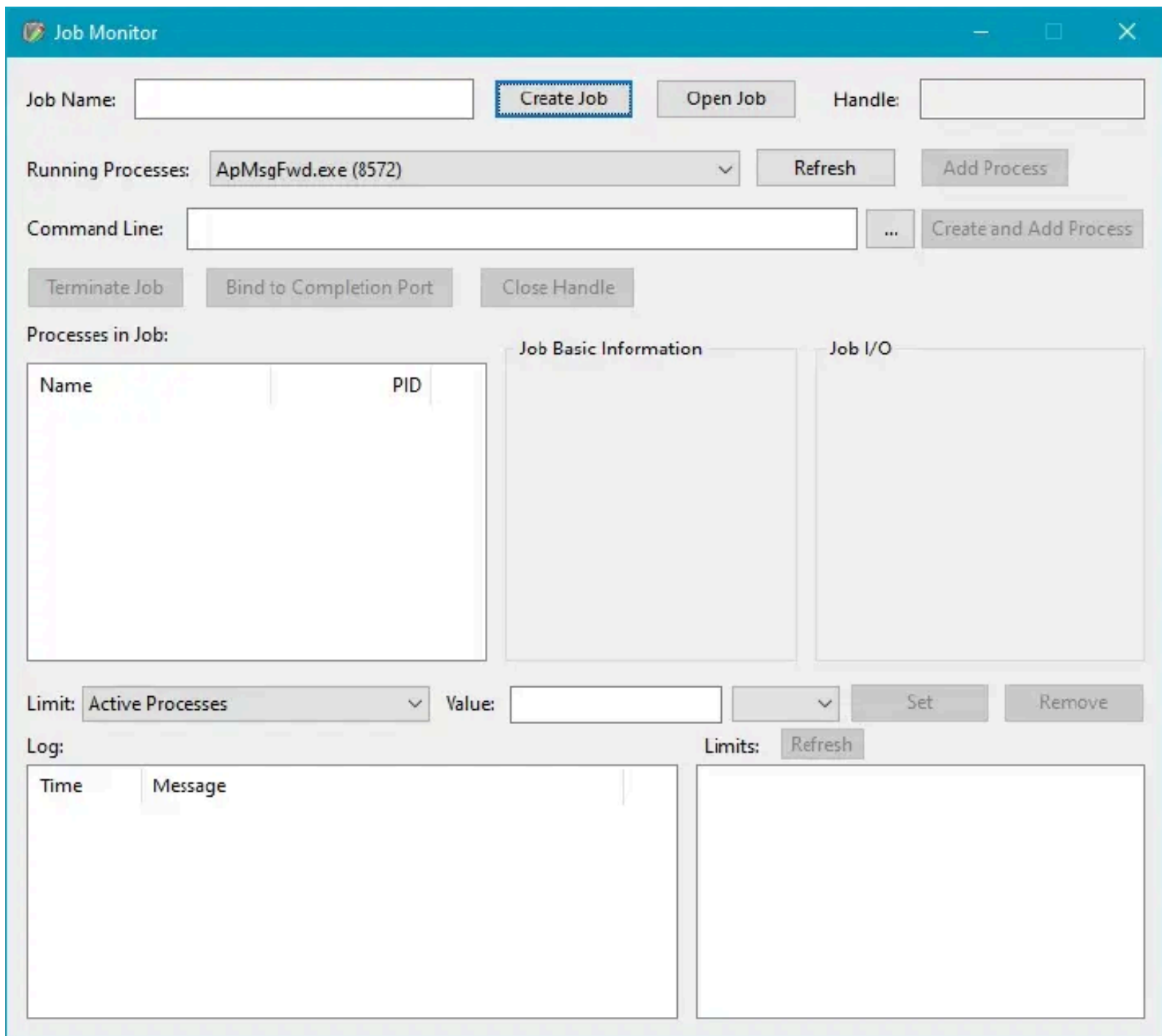
Similarly, using `JobObjectBasicAndIoAccountingInformation` and `JOBOBJECT_BASIC_AND_IO_ACCOUNTING_INFORMATION` provides extended accounting information for a job, that includes I/O operations count and sizes. This extended structure includes two structures, one of which is `JOBOBJECT_BASIC_ACCOUNTING_INFORMATION`:

```
typedef struct _IO_COUNTERS {
    ULONGLONG ReadOperationCount;
    ULONGLONG WriteOperationCount;
    ULONGLONG OtherOperationCount;
    ULONGLONG ReadTransferCount;
    ULONGLONG WriteTransferCount;
    ULONGLONG OtherTransferCount;
} IO_COUNTERS, *PIO_COUNTERS;

typedef struct JOBOBJECT_BASIC_AND_IO_ACCOUNTING_INFORMATION {
    JOBOBJECT_BASIC_ACCOUNTING_INFORMATION BasicInfo;
    IO_COUNTERS IoInfo;
} JOBOBJECT_BASIC_AND_IO_ACCOUNTING_INFORMATION, *PJOBOBJECT_BASIC_AND_IO_ACCOUNTING_INFORMATION;
```

Read and write operations refer to `ReadFile` and `WriteFile` (and similar) APIs, which we look at later in this book. The “other” operations refers to usage of the `DeviceIoControl` API, that is issued in non-read/write operations, typically targeting devices rather than file system files.

The *JobMon* project, part of this chapter’s source code includes many of the features of Jobs that we discuss in this chapter. Running it shows the window in figure 4-8.

Figure 4-8: *Job Monitor* initial window

Click on the *Create Job* button to create an empty job. You can set a name for job before creating the job. The job is created with zero processes and shows the basic and I/O information (figure 4-9).

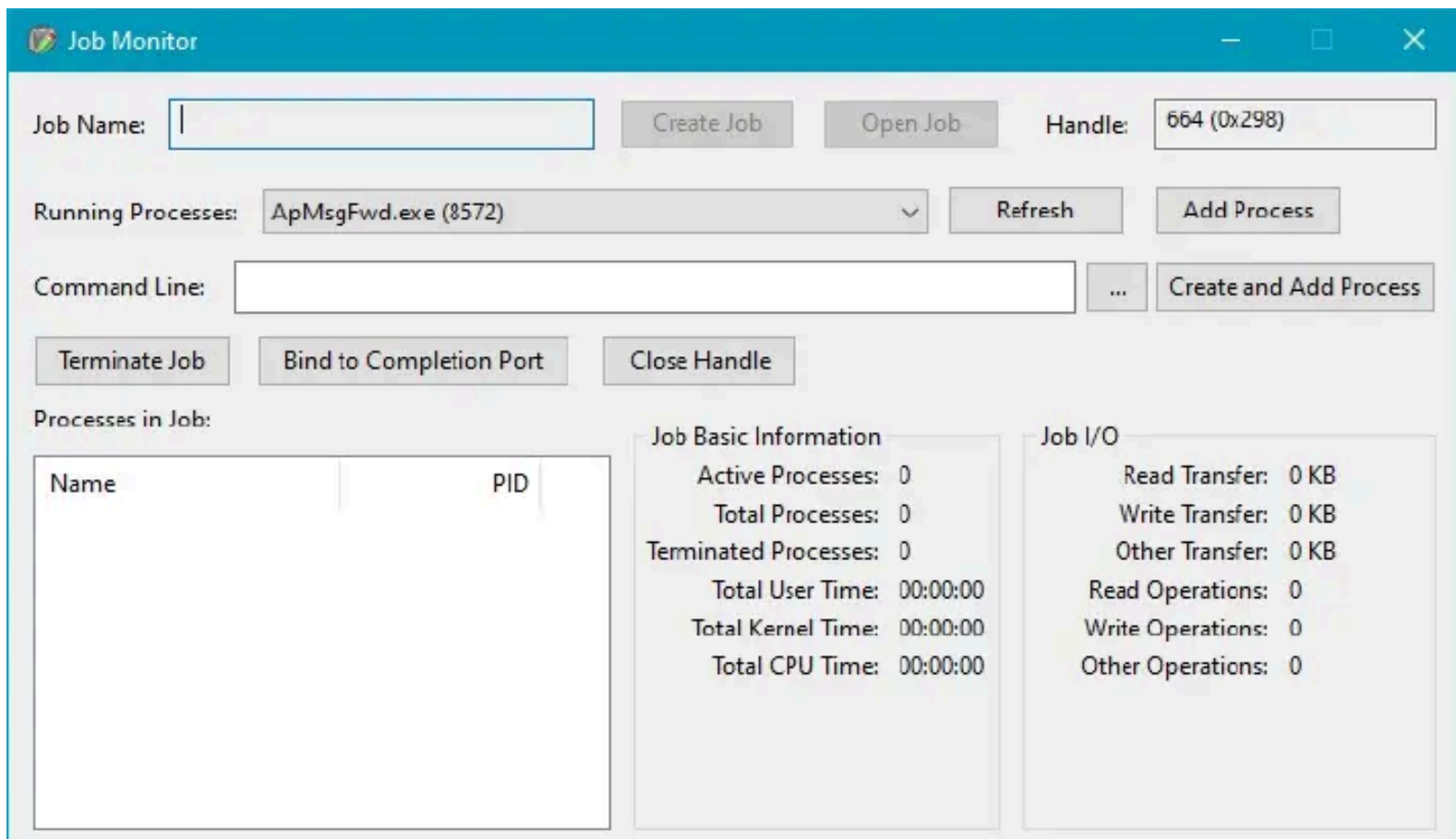
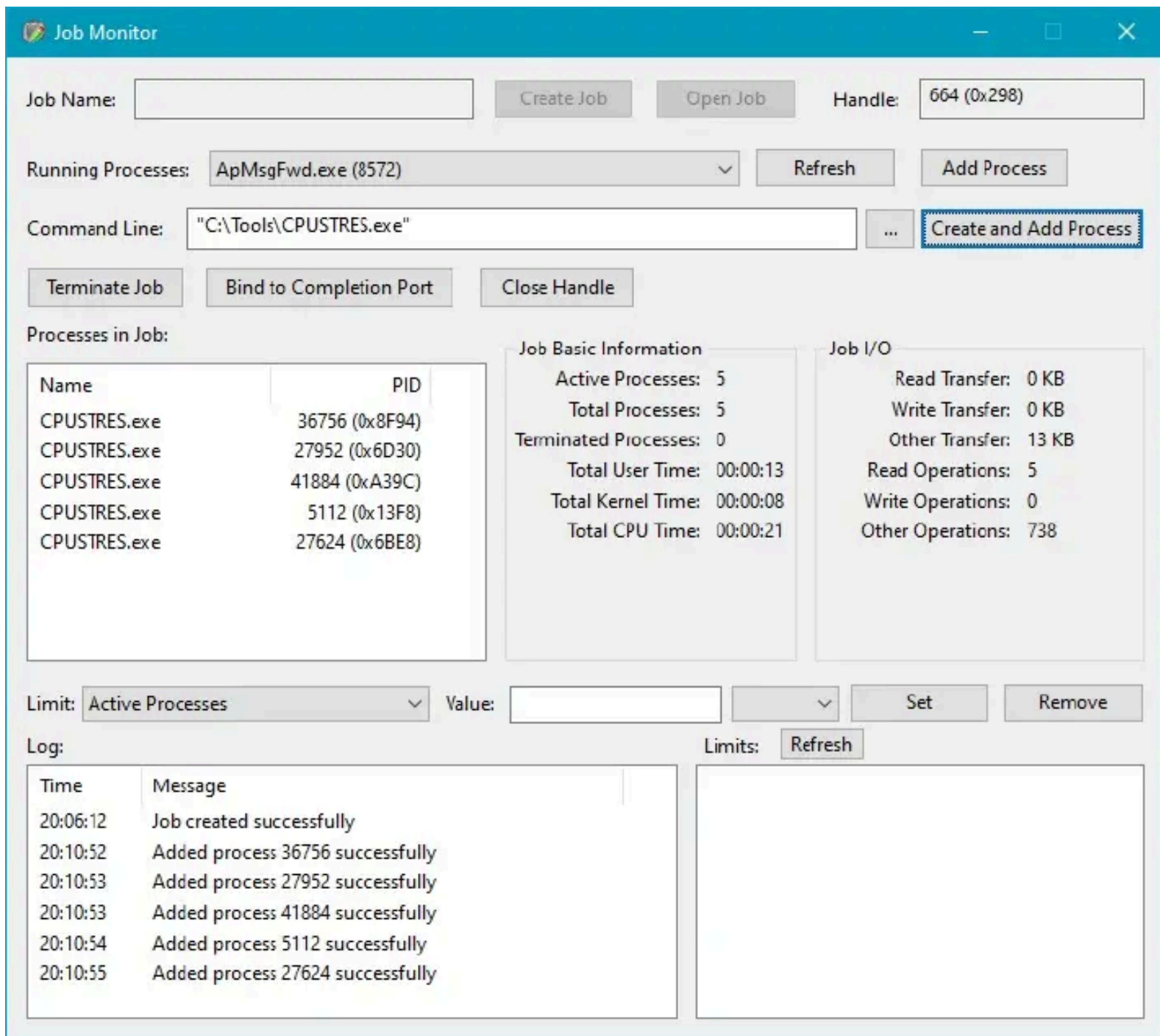
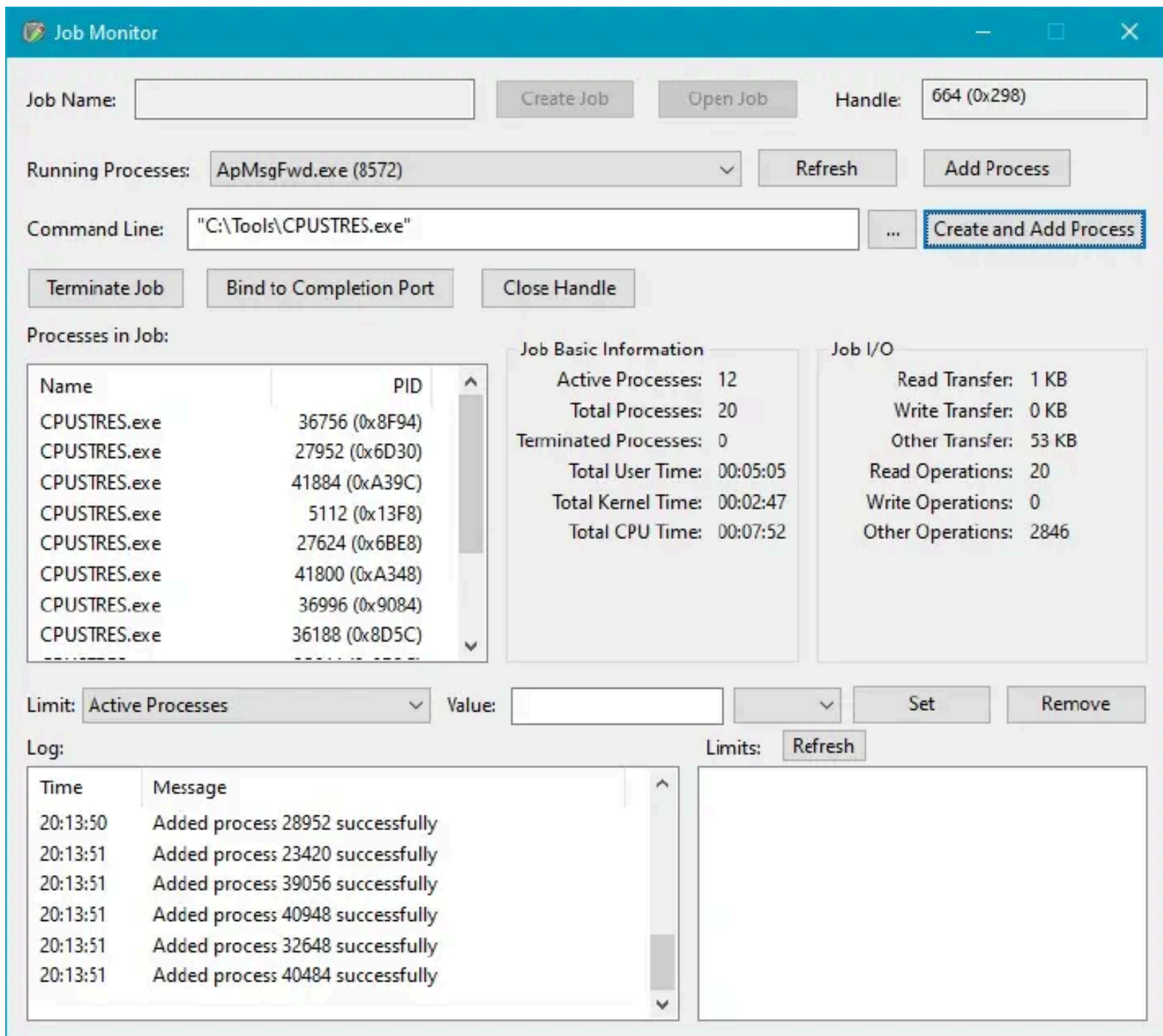


Figure 4-9: *Job Monitor* with a newly created job

To see job accounting in action, download the *CPUSres.exe* tool from <https://github.com/zodiacon/AllTools>. Click the three dots button to browse for *CpuStres.exe*. Then click *Create and Add Process* button several times to add instances of *CPUSres* to the job (figure 4-10). Notice the accounting information is no longer zero.

Figure 4-10: *Job Monitor* with a several *CPUSTres* processes

CPUSTres is a CPU-eating utility, which is used more in the next chapter. The display is updated roughly every 1.5 seconds. You can add more processes to the job (either *CPUSTres* or another image), and close processes. Figure 4-11 shows *Job Monitor* after more *CPUSTres* are added with some closed.

Figure 4-11: *Job Monitor* with more processes

You can click *Terminate Job* to terminate all processes in the job in one stroke. This is achieved by calling `TerminateJobObject`:

```
BOOL TerminateJobObject(
    _In_ HANDLE hJob,
    _In_ UINT uExitCode);
```

`TerminateJobObject` behaves as if every process active in the job is terminated with `TerminateProcess` where `uExitCode` is the exit code of all processes in the job.

At this point, new processes may be added to the job, with accounting information updated normally.

Querying for Job Process List

The list of active (live) processes in a job can be retrieved by calling `QueryInformation-JobObject` with the `JobObjectBasicProcessIdList` information class, returning an array of process IDs in the `JOBOBJECT_BASIC_PROCESS_ID_LIST` structure:

```
typedef struct _JOBJECT_BASIC_PROCESS_ID_LIST {
    DWORD NumberOfAssignedProcesses;
    DWORD NumberOfProcessIdsInList;
    ULONG_PTR ProcessIdList[1];
} JOBJECT_BASIC_PROCESS_ID_LIST, *PJOBJECT_BASIC_PROCESS_ID_LIST;
```

The structure has a variable size because of the array of process IDs. This means a fixed size array should be large enough to include all process IDs and hope for the best. Alternatively, a dynamically-allocated buffer can be used and its size adjusted if it's not large enough. The following example shows how to retrieve the list of active processes while allocating large enough buffer as required.

```
#include <vector>
#include <memory>

std::vector<DWORD> GetJobProcessList(HANDLE hJob) {
    auto size = 256;
    std::vector<DWORD> pids;
    while (true) {
        auto buffer = std::make_unique<BYTE[]>(size);
        auto ok = ::QueryInformationJobObject(hJob, JobObjectBasicProcessIdList\
,
        buffer.get(), size, nullptr);
        if (!ok && ::GetLastError() == ERROR_MORE_DATA) {
            // buffer too small - resize and try again
            size *= 2;
            continue;
        }
        if (!ok)
            break;

        auto info = reinterpret_cast<JOBJECT_BASIC_PROCESS_ID_LIST*>(buffer.g\
```

```
et());  
    pids.reserve(info->NumberOfAssignedProcesses);  
    for (DWORD i = 0; i < info->NumberOfAssignedProcesses; i++)  
        pids.push_back((DWORD)info->ProcessIdList[i]);  
    break;  
}  
return pids;  
}
```

The code uses some C++ constructs to simplify memory management. The function itself returns a `std::vector<DWORD>` holding the process IDs in the job. In the most general case the number of processes is not known in advance, so the function allocates a buffer with `std::make_unique<BYTE>[]`, which allocates a byte array with the given number of elements (`size`). The `unique_ptr`'s destructor frees the buffer when it goes out of scope.

Next, `QueryInformationJobObject` is called with the allocated byte buffer. If it returns `FALSE` and `GetLastError` returns `ERROR_MORE_DATA`, it means the allocated buffer is too small, so the function doubles `size` and tries again.

Once the buffer is large enough, the pointer is cast to `JOB_OBJECT_BASIC_PROCESS_ID_LIST*`, and the process IDs can be retrieved and placed into the `std::vector`. Curiously enough, the process IDs returned in this structure are typed as `ULONG_PTR`, which means each one is 64 bit in a 64 bit process. This is unusual, as process IDs are normally 32 bit values (`DWORD`). This is why we cannot simply copy the entire array in one stroke to the vector (unless the vector is changed to hold `ULONG_PTR`s).

You may be curious why the process IDs are typed as `ULONG_PTR`. This is one of the very few cases in the Windows API where this occurs. Within the kernel, process (and thread) IDs are generated using a private handle table for just this purpose. And since handles on 64 bit systems are 64 bit values, it may be “naturally” easy to use them as is. Still, since handle tables are limited to about 16 million handles, 64 bit values are not currently needed (and not used for process IDs outside the kernel).

Setting Job Limits

The primary purpose of a job is to place limits on its processes. The function to use is the opposite of `QueryInformationJobObject - SetInformationJobObject` defined like so:

```
BOOL SetInformationJobObject(
    _In_ HANDLE          hJob,
    _In_ JOBOBJECTINFOCLASS JobObjectInfoClass,
    _In_ PVOID           pJobObjectInfo,
    _In_ DWORD           cbJobObjectInfoLength);
```

The arguments should be self-explanatory at this point. The job handle must have the `JOB_OBJECT_SET_ATTRIBUTES` access mask, and cannot be `NULL`. Table 4-3 summarizes the (documented) information classes that can be used with `SetInformationJobObject`.

Table 4-3: JOBOBJECTINFOCLASS for documented job set operations

Information class (JobObject-)	Information structure	Description
BasicLimitInformation (2)	JOBOBJECT_BASIC_LIMIT_INFORMATION	Basic limits
BasicUIRestrictions (4)	JOBOBJECT_BASIC_UI_RESTRICTIONS	User interface limits
EndOfJobTimeInformation (6)	JOBOBJECT_END_OF_JOB_TIME_INFORMATION	What happens when the end of job time limit is reached
AssociateCompletionPortInformation (7)	JOBOBJECT_ASSOCIATE_COMPLETION_PORT	Associate a completion port with the job
ExtendedLimitInformation (9)	JOBOBJECT_EXTENDED_LIMIT_INFORMATION	Extended limits
GroupInformation (11)	USHORT array	(Windows 7+) Processor groups for the job (see chapter 6)
GroupInformationEx (14)	GROUP_AFFINITY array	(Windows 8+) Processor groups and affinities for the job (see chapter 6)
NotificationLimitInformation (12)	JOBOBJECT_NOTIFICATION_LIMIT_INFORMATION	(Windows 8+) Notification limits
LimitViolationInformation (13)	JOBOBJECT_LIMIT_VIOLATION_INFORMATION	(Windows 8+) Information on limits violations
GroupInformationEx (14)	GROUP_AFFINITY array	(Windows 8+) Processor group affinity

Table 4-3: JOBOINFOCLASS for documented job set operations

Information class (JobObject-)	Information structure	Description
CpuRateControlInformation (15)	JOBOBJECT_CPU_RATE_- CONTROL_INFORMATION	(Windows 8+) CPU rate limit information
NetRateControlInformation (32)	JOBOBJECT_NET_RATE_- CONTROL_INFORMATION	(Windows 10+) Network rate limit information
NotificationLimitInformation (33)	JOBOBJECT_NOTIFICATION_- LIMIT_INFORMATION_2	(Windows 10+) Extended limit information
LimitViolationInformation2 (34)	JOBOBJECT_LIMIT_- VIOLATION_INFORMATION_2	(Windows 10+) Extended limit violation information

The most “fundamental” limits are specified with `JobObjectBasicLimitInformation` and `JOBOBJECT_BASIC_LIMIT_INFORMATION`, while extended limits are set with `JobObjectExtendedLimitInformation` and `JOBOBJECT_EXTENDED_LIMIT_INFORMATION`. These structures are defined like so:

```
typedef struct _JOBOBJECT_BASIC_LIMIT_INFORMATION {
    LARGE_INTEGER PerProcessUserTimeLimit;
    LARGE_INTEGER PerJobUserTimeLimit;
    DWORD LimitFlags;
    SIZE_T MinimumWorkingSetSize;
    SIZE_T MaximumWorkingSetSize;
    DWORD ActiveProcessLimit;
    ULONG_PTR Affinity;
    DWORD PriorityClass;
    DWORD SchedulingClass;
} JOBOBJECT_BASIC_LIMIT_INFORMATION, *PJOBOBJECT_BASIC_LIMIT_INFORMATION;
```

```
typedef struct _JOBOBJECT_EXTENDED_LIMIT_INFORMATION {
    JOBOBJECT_BASIC_LIMIT_INFORMATION BasicLimitInformation;
    IO_COUNTERS IoInfo;
    SIZE_T ProcessMemoryLimit;
    SIZE_T JobMemoryLimit;
    SIZE_T PeakProcessMemoryUsed;
    SIZE_T PeakJobMemoryUsed;
} JOBOBJECT_EXTENDED_LIMIT_INFORMATION, *PJOBOBJECT_EXTENDED_LIMIT_INFORMATION;
```

The various limits that can be set depend on the `LimitFlags` member in `JOBOBJECT_BA-`

SIC_LIMIT_INFORMATION (whether standalone or part of JOBOBJECT_EXTENDED_LIMIT_INFORMATION). Some flags have no associated member, as these flags themselves are enough. Others make the corresponding member's value used by SetInformationJobObject. Table 4-4 summarizes the flags that have no corresponding member. Table 4-5 summarizes the flags that have associated member(s).

Table 4-4: LimitFlags with no associated member

Limit flag (JOB_OBJECT_LIMIT_*)	Description
DIE_ON_UNHANDLED_EXCEPTION (0x400)	Prevents processes in the job from showing a dialog box in case of an unhandled exception
PRESERVE_JOB_TIME (0x40)	This flag preserves any previously set job limits, so that the caller can just make further changes. This flag cannot be used with JOB_OBJECT_LIMIT_JOB_TIME (and vice versa)
BREAKAWAY_OK (0x800)	Allows processes created from a process in the job to be created outside the job if CREATE_BREAKAWAY_FROM_JOB flag is specified in the call to CreateProcess. If the process is part of a job hierarchy, the new process breaks from this job and all parent jobs that have this flag set
SILENT_BREAKAWAY_OK (0x1000)	Allows processes created from a process in the job to be created outside of the job without any special flag to CreateProcess
KILL_ON_JOB_CLOSE (0x2000)	Terminate all processes in the job when the last job handle is closed

Table 4-5: LimitFlags with associated member(s)

Limit flag (JOB_OBJECT_LIMIT_*)	Associated member (B/E)	Description
WORKINGSET (1)	MinimumWorkingSetSize, MaximumWorkingSetSize (B)	Limits the per-process working set (RAM). If the process needs to use more RAM, it will page towards itself
PROCESS_TIME (2)	PerProcessUserTimeLimit (B)	Limits the user mode execution time of each process in the job (in 100 nano-sec units)

Table 4-5: `LimitFlags` with associated member(s)

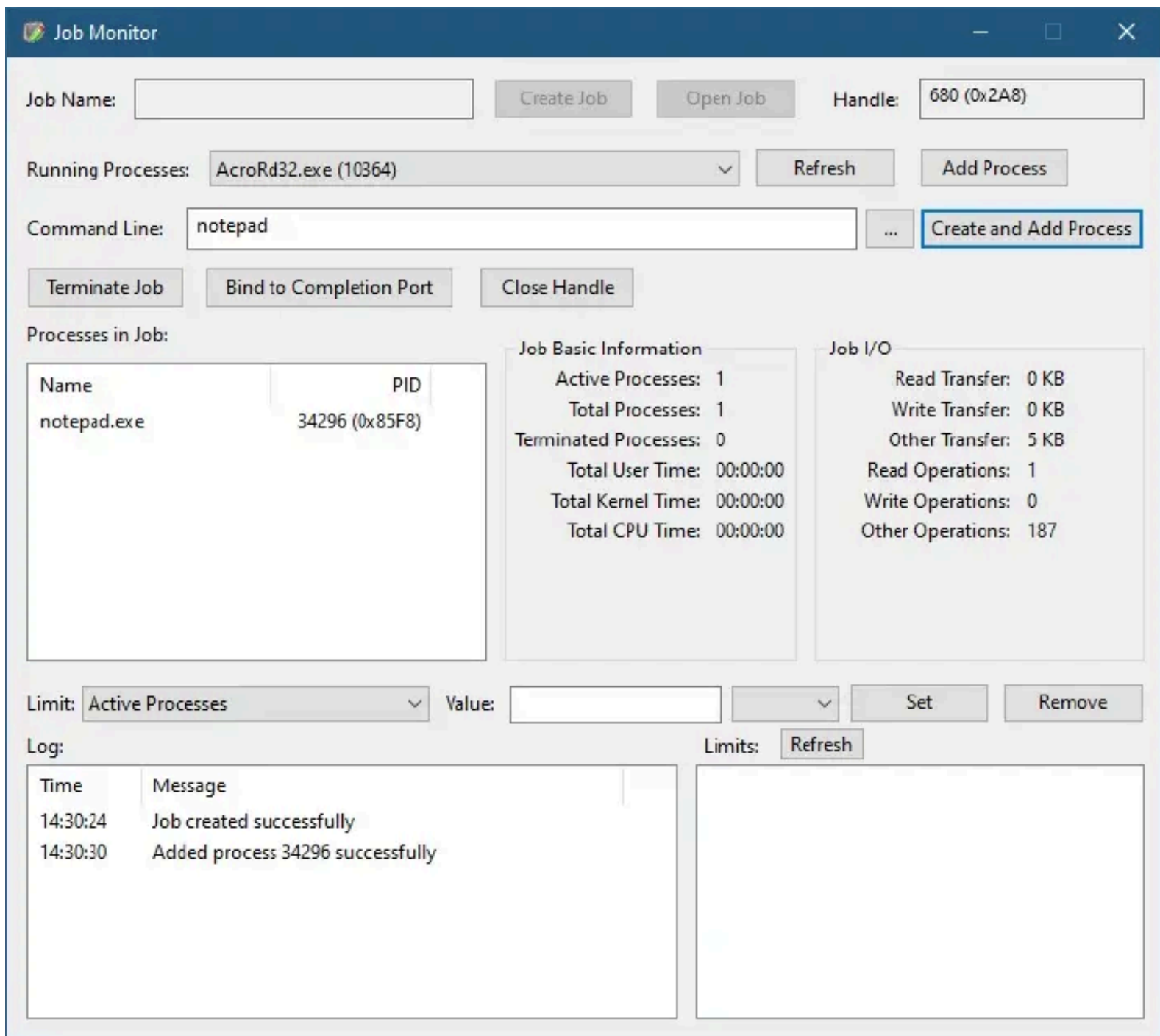
Limit flag (<code>JOB_OBJECT_LIMIT_*</code>)	Associated member (B/E)	Description
<code>JOB_TIME</code> (4)	<code>PerJobUserTimeLimit</code> (B)	Limits the job-wide user-mode CPU time. If the process uses more CPU time, it is terminated
<code>ACTIVE_PROCESS</code> (8)	<code>ActiveProcessLimit</code> (B)	Limits the active (live) processes in the job. Newly created processes that violate this limit are automatically terminated
<code>AFFINITY</code> (0x10)	<code>Affinity</code> (B)	Sets the CPU affinity for all processes in the job (see chapter 6 for more on affinity)
<code>PRIORITY_CLASS</code> (0x20)	<code>PriorityClass</code> (B)	Limits the processes in the job to use the same priority class (see chapter 6)
<code>SCHEDULING_CLASS</code> (0x80)	<code>SchedulingClass</code> (B)	Limits the scheduling class for all processes in the job. The values are 0 to 9, where 9 is the highest. The default is 5 (see chapter 6 for more information)
<code>PROCESS_MEMORY</code> (0x100)	<code>JobMemoryLimit</code> (E)	Limits the per-process committed memory
<code>JOB_MEMORY</code> (0x200)	<code>JobMemoryLimit</code> (E)	Limits the job-wide committed memory
<code>SUBSET_AFFINITY</code> (0x4000)	<code>Affinity</code> (B)	(Windows 7+) Allows processes to use a subset of the affinity specified. The <code>JOB_OBJECT_LIMIT_AFFINITY</code> flag is required as well

All the flags in table 4-4 must be used with the extended limits structure (`JOB_OBJECT_EXTENDED_LIMIT_INFORMATION`), where the flags themselves are specified with the nested `JOB_OBJECT_BASIC_LIMIT_INFORMATION` structures' `LimitFlags` member. In table 4-5, *B/E* indicates whether this limit is specified with the basic (B) or the extended (E) structure.

The following code sets a priority class of *Below Normal* for the given job:

```
bool SetJobPriorityClass(HANDLE hJob) {
    JOBOBJECT_BASIC_LIMIT_INFORMATION info;
    info.LimitFlags = JOB_OBJECT_LIMIT_PRIORITY_CLASS;
    info.PriorityClass = BELOW_NORMAL_PRIORITY_CLASS;
    return ::SetInformationJobObject(hJob, JobObjectBasicLimitInformation,
        &info, sizeof(info));
}
```

We can test this type of functionality with the *Job Monitor* application. Open *Job Monitor*, create a new job, and add a process to the job (*Notepad* in figure 4-12).

Figure 4-12: *Job Monitor* with *Notepad* added

If you examine the *Base Priority* column in *Task Manager* for this *Notepad* process, you should see the value *Normal* (figure 4-13).

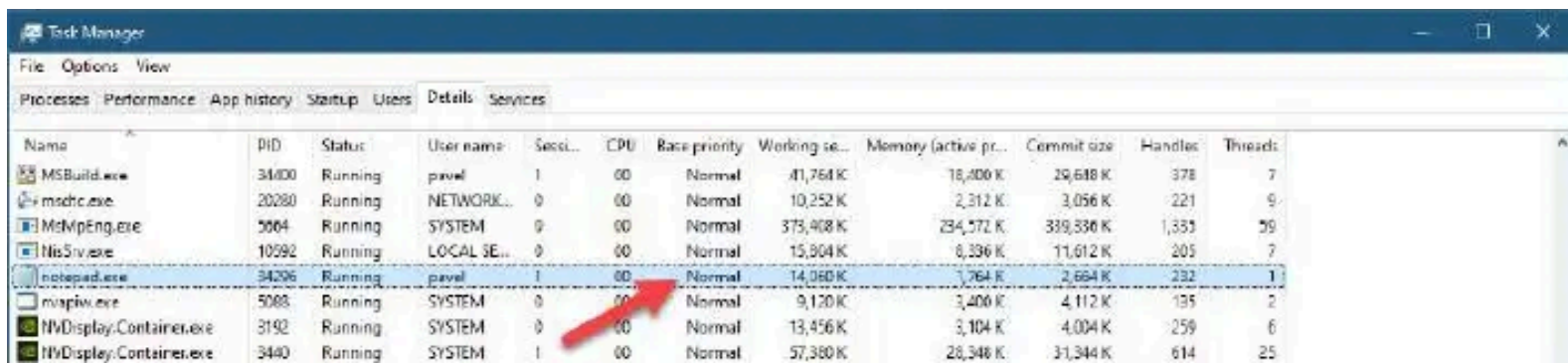


Figure 4-13: Base Priority of Notepad in Task Manager

The exact meaning and effect of Base Priority (Priority Class) is discussed in chapter 6.

Now return to *Job Monitor*, select the *Priority Class* limit and set its value to *Below Normal* and click *Set* (figure 4-14).

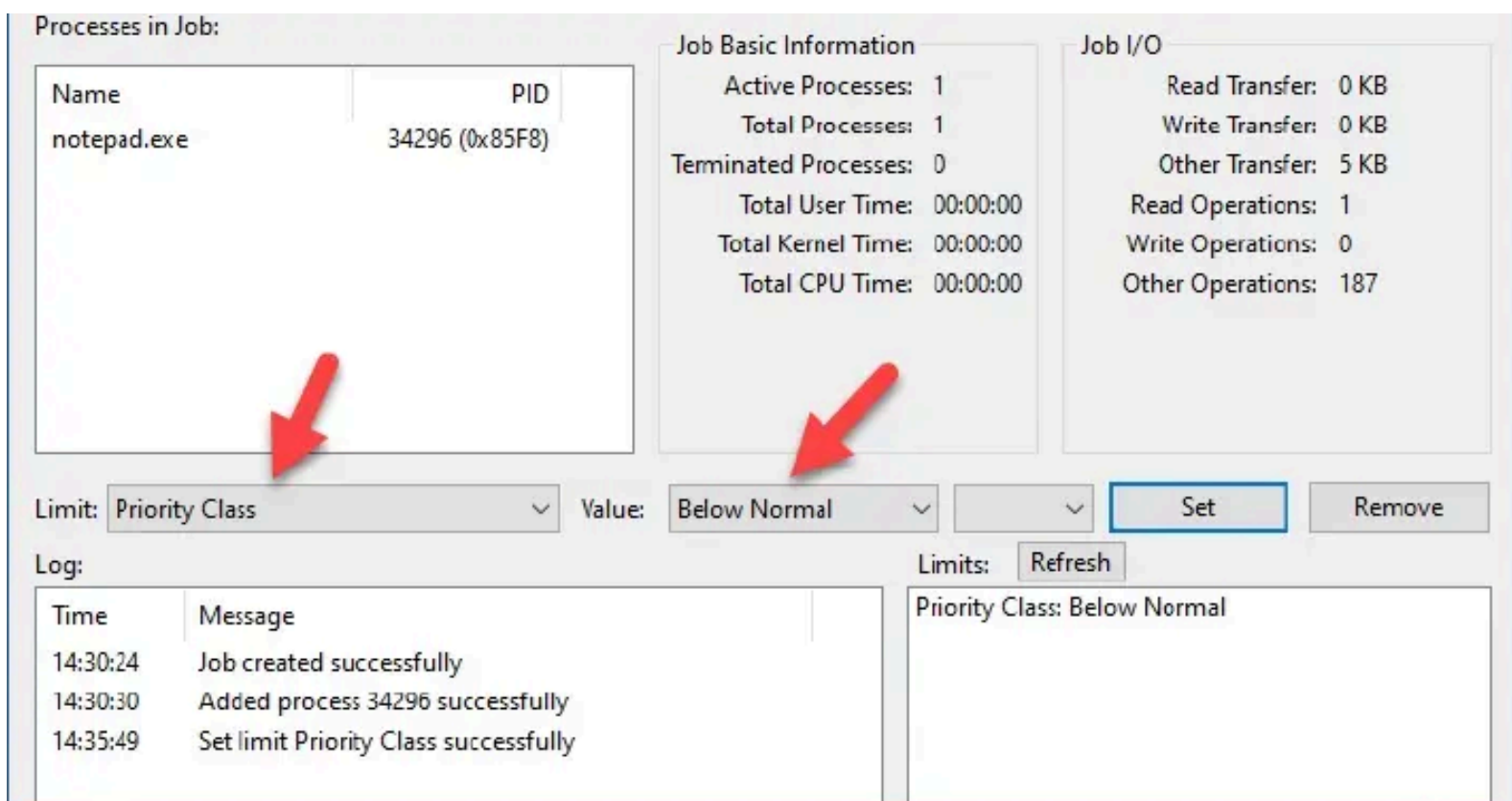
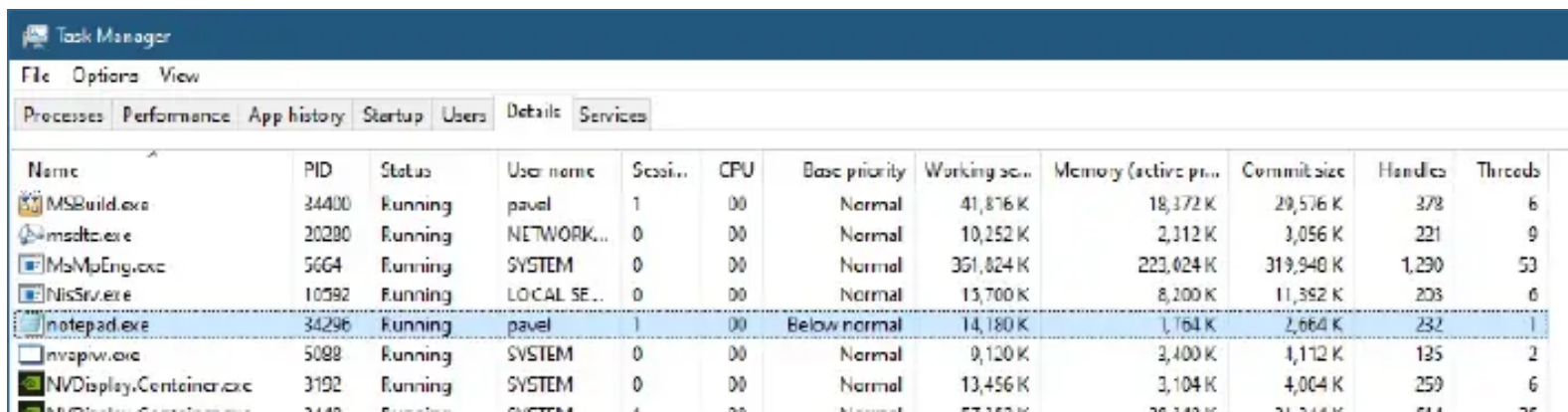


Figure 4-14: Setting Priority Class limit in Job Monitor

Now switch to *Task Manager*. *Notepad*'s base priority should now show *Below Normal* (figure 4-15).



Name	PID	Status	User name	Session	CPU	Base priority	Working set	Memory (active pr...	Commit size	Handles	Threads
MSBuild.exe	34400	Running	pavel	1	00	Normal	41,876 K	18,372 K	29,576 K	378	6
msmdtc.exe	20280	Running	NETWORK...	0	00	Normal	10,152 K	2,112 K	3,056 K	221	9
MsMpEng.exe	5664	Running	SYSTEM	0	00	Normal	351,824 K	223,024 K	319,940 K	1,290	53
NisSrv.exe	10592	Running	LOCAL SE...	0	00	Normal	13,700 K	8,200 K	11,392 K	203	6
notepad.exe	34296	Running	pavel	1	00	Below normal	14,180 K	1,764 K	2,664 K	232	1
nvapw.exe	5088	Running	SYSTEM	0	00	Normal	9,120 K	3,400 K	4,112 K	135	2
NVDisplay.Container.exe	3192	Running	SYSTEM	0	00	Normal	13,456 K	3,104 K	4,004 K	259	6

Figure 4-15: Limited base priority of *Notepad*

Trying to change the base priority with *Task Manager* by right-clicking *Notepad* and selecting *Set Priority* with any level (except *Below Normal*) will have no effect because of the job limit.

Going back to *Job Monitor* and clicking *Remove* on the priority class limit will re-allow base priority modifications.



You can find the code to set/remove most of the available job limits in the *JobMon* project in the *MainDlg.cpp* file.

CPU Rate Limit

Windows 8 added a CPU rate limit to the available job limits, which is not set by the basic or extended limits, but rather uses its own job limit enumeration: `JobObjectCpuRateControlInformation`.

The associated structure is `JOB_OBJECT_CPU_RATE_CONTROL_INFORMATION` defined like so:

```
typedef struct _JOB_OBJECT_CPU_RATE_CONTROL_INFORMATION {
    DWORD ControlFlags;
    union {
        DWORD CpuRate;
        DWORD Weight;
        struct {
            WORD MinRate;
            WORD MaxRate;
        };
    };
};
} JOB_OBJECT_CPU_RATE_CONTROL_INFORMATION, *PJOB_OBJECT_CPU_RATE_CONTROL_INFORMATION;
```

There are three different ways to set CPU rate limits, controlled by the `ControlFlags` field. Its possible values are summarized in table 4-5.

Table 4-5: Flags for CPU rate control

Flag (<code>JOB_OBJECT_CPU_RATE_*</code>)	Description
ENABLE (1)	Enable CPU rate control
WEIGHT_BASED (2)	CPU rate is based on relative weight (<code>Weight</code> member)
HARD_CAP (4)	Sets a hard cap for CPU consumption
NOTIFY (8)	Notifies the I/O completion port associated with the job (if any) for rate violations
MIN_MAX_RATE (0x10)	Sets the CPU rate between minimum and maximum values (<code>MinRate</code> and <code>MaxRate</code> members)

If CPU rate control is enabled, and neither `JOB_OBJECT_CPU_RATE_CONTROL_WEIGHT_BASED` nor `JOB_OBJECT_CPU_RATE_CONTROL_MIN_MAX_RATE` are specified, then the `CpuRate` member specifies the CPU limit percentage relative to 10000. For example, if 15 percent CPU is desired, the value should be set to 1500. This allows fractional CPU rates to be specified.

If the `JOB_OBJECT_CPU_RATE_CONTROL_HARD_CAP` flag is also specified, the limit is a hard one - the job won't get more CPU even if there are CPUs available. Without this flag, the job may get more CPU time if there are available processors.

Behind the scenes, the kernel applied these restrictions by measuring the CPU consumption of the job in 300 msec time intervals, allowing/preventing it to/from executing in the next interval(s).

The *CpuLimit* project demonstrates using CPU rate control with the `CpuRate` member and a hard cap. The main function accepts an array of process IDs to put in a job, and a percentage to use as a hard CPU rate limit.

Here is the start of main:

```

int main(int argc, const char* argv[]) {
    if (!::IsWindows8OrGreater()) {
        printf("CPU Rate control is only available on Windows 8 and later\n");
        return 1;
    }

    if (argc < 3) {
        printf("Usage: CpuLimit <pid> [<pid> ...] <percentage>\n");
        return 0;
    }

    // create the job object

    HANDLE hJob = ::CreateJobObject(nullptr, L"CpuRateJob");
    if (!hJob)
        return Error("Failed to create object");

    for (int i = 1; i < argc - 1; i++) {
        int pid = atoi(argv[i]);
        HANDLE hProcess = ::OpenProcess(PROCESS_SET_QUOTA | PROCESS_TERMINATE,
            FALSE, pid);
        if (!hProcess) {
            printf("Failed to open handle to process %d (error=%d)\n", pid,
                ::GetLastError());
            continue;
        }
        if (!::AssignProcessToJobObject(hJob, hProcess)) {
            printf("Failed to assign process %d to job (error=%d)\n", pid,
                ::GetLastError());
        }
        else {
            printf("Added process %d to job\n", pid);
        }
        ::CloseHandle(hProcess);
    }
}

```

main starts by checking whether the application is executing on Windows 8 at least, as CPU rate control was not available in prior versions. Then, the program validates that there are at least one process ID and a CPU percentage by checking for at least 3 arguments (the program

itself, PID, rate).

A job object is then created, with a name, so it's easier to identify using tools. Each process is opened and assigned to the job, if possible.

Now the program is ready to apply the CPU rate control:

```

JOB_OBJECT_CPU_RATE_CONTROL_INFORMATION info;
info.CpuRate = atoi(argv[argc - 1]) * 100;
info.ControlFlags = JOB_OBJECT_CPU_RATE_CONTROL_ENABLE |
    JOB_OBJECT_CPU_RATE_CONTROL_HARD_CAP;

if (!::SetInformationJobObject(hJob, JobObjectCpuRateControlInformation,
    &info, sizeof(info)))
    return Error("Failed to set job limits");

printf("CPU limit set successfully.\n");

printf("Press ENTER to quit.\n");
char dummy[10];
gets_s(dummy);

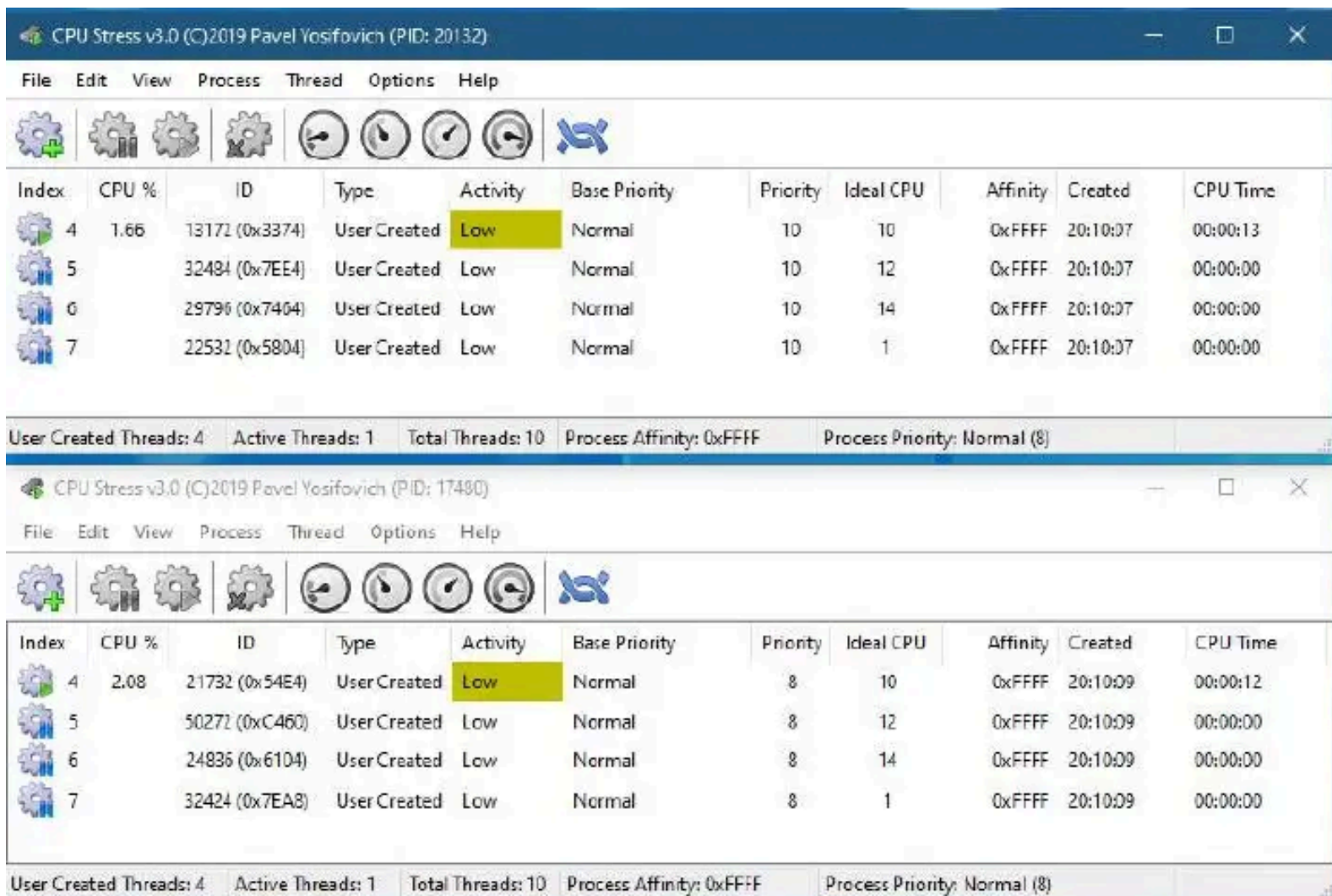
::CloseHandle(hJob);
return 0;
}

```

The CPU rate limit is calculated by taking the argument from the command line and multiplying it by 100. Finally, `SetInformationJobObject` is called to set the limit.

Before the program exits, it waits for the user to press *ENTER*. This allows the job object's handle to be open, and so easier to spot with tools. Otherwise, the handle would have been closed, marking the job for deletion. The limits, however, are still applied, as long as there are processes active in the job.

Let's test this by launching two instances of the *CpuStress* application (figure 4-16). The system used here has 16 logical processors, so we'll activate 4 threads with maximum activity in both of them. That should consume about 50% of all CPU time on the system (figure 4-17). *Task Manager* shows this is indeed the case (figure 4-18).

Figure 4-16: 2 instances of *CPU Stress* when launched

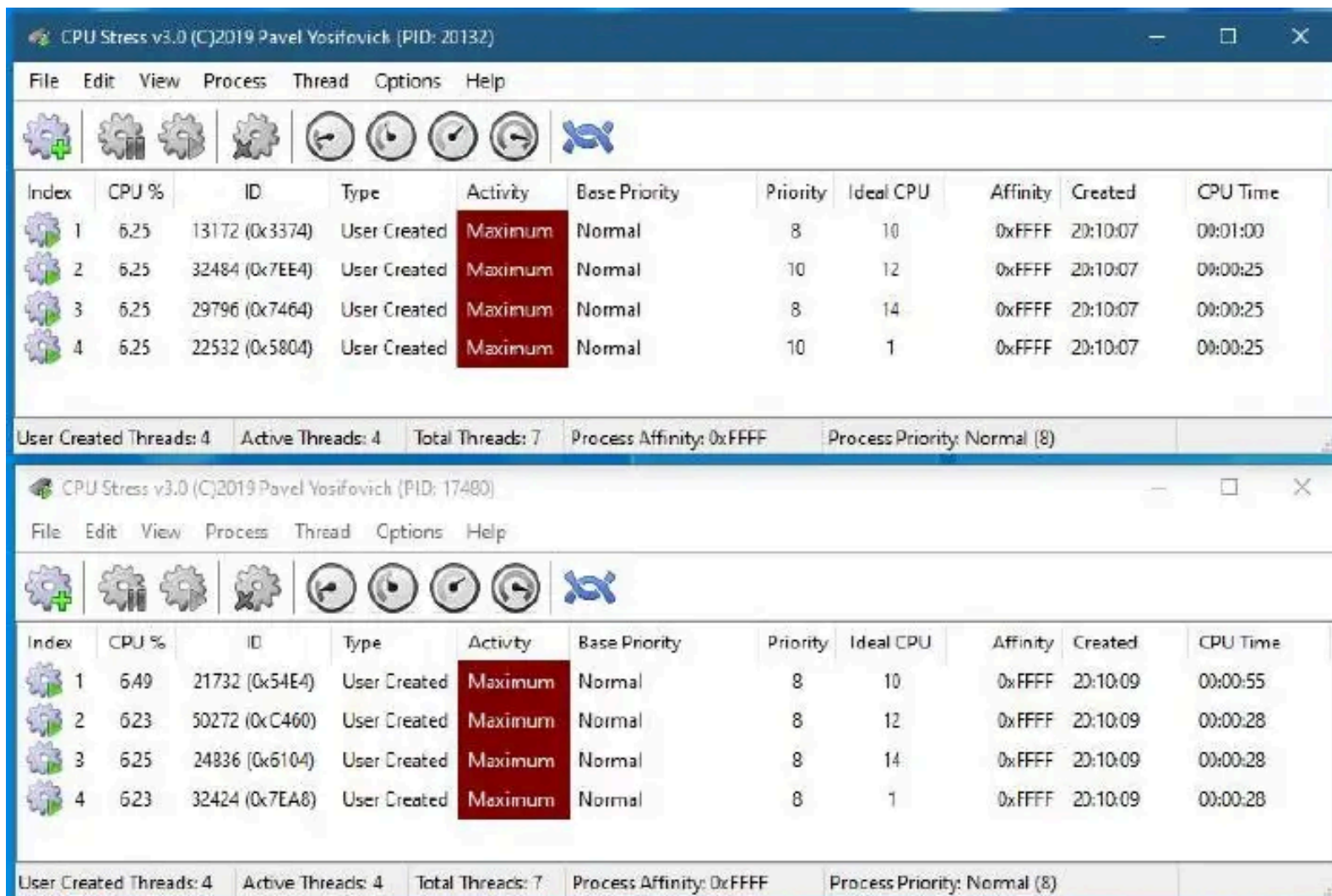


Figure 4-17: 2 CPU Stress with 4 maximum active threads

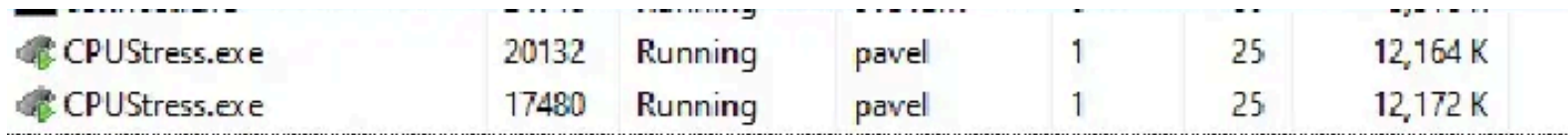


Figure 4-18: Task Manager showing 50% CPU utilization

Now we execute *CpuLimit* with the process IDs and the required CPU rate limit (20% in this example):

```
cpulimit 38984 28760 20
```

You should see a set of success messages like so:

```
CpuLimit.exe 20132 17480 20
Added process 20132 to job
Added process 17480 to job
CPU limit set successfully.
Press ENTER to quit.
```

At this point, you should be able to see the CPU consumption drop in both *CPUSStress* instances (figure 4-19). The total CPU both consume should be around 20%, viewable in *Task Manager*.

Opening *Job Explorer* and looking at the *CpuRateJob* job (this is the name given in the code for easy identification) should show the CPU rate limit (figure 4-20).



Unfortunately, at the time of this writing, *Process Explorer* does not show CPU rate control information for jobs.

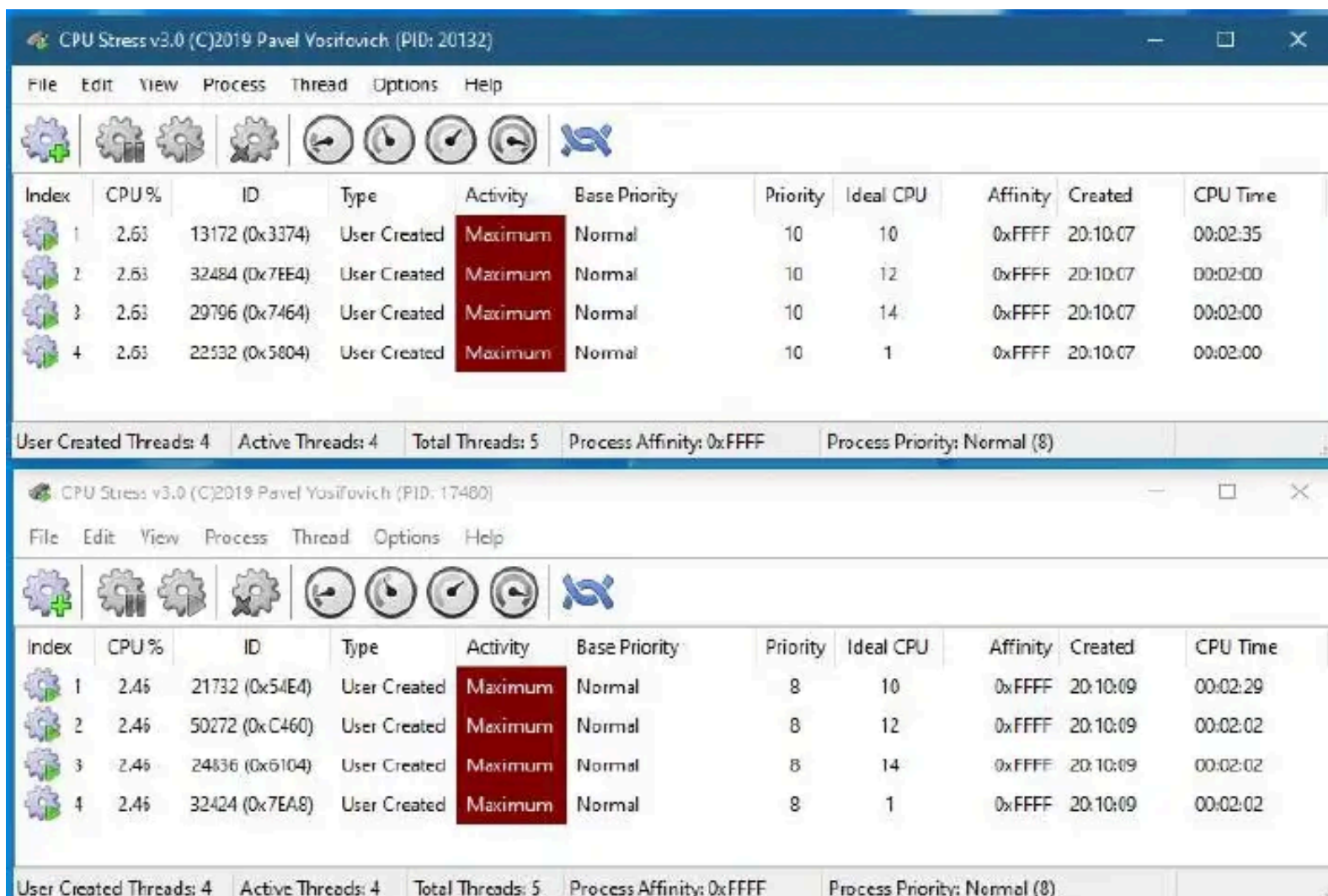


Figure 4-19: *CPU Stress* instances affected by CPU rate control

General		
Address:	0xFFFFC78D896EC770	
Name:	\Sessions\1\BaseNamedObjects\CpuRateJob	
Active Processes:	2	
Total Processes:	2	
User Time:	00:04:32.734	
Kernel Time:	00:18:09.890	
CPU Time:	00:22:42.625	
Terminated Processes:	0	
Page Faults:	8430	
Silo:		
Processes		
CPUStress.exe	PID: 20132 (0x4EA4)	Created: 11/18/19 20:10:07.072
CPUStress.exe	PID: 17480 (0x4448)	Created: 11/18/19 20:10:09.530
Limits		
CPU Rate	Rate: 20.00% (Hard Cap)	
Queue Handler		

Figure 4-20: *Job Explorer* showing CPU rate control

If *CpuLimit* fails to add the processes with error 5 (access denied), run *CpuStress* from a command window rather than through explorer. If you're curious, investigate why this happens.

User Interface Limits

Another set of job limits is available through the `JobObjectBasicUIRestrictions` information class for user interface related restrictions. These are represented by a single 32-bit value stored in a simple structure:

```
typedef struct _JOBOBJECT_BASIC_UI_RESTRICTIONS {
    DWORD UIRestrictionsClass;
} JOBOBJECT_BASIC_UI_RESTRICTIONS, *PJOBOBJECT_BASIC_UI_RESTRICTIONS;
```

The available restrictions are bit flags listed in table 4-6.



A job that uses UI restrictions cannot be part of a job hierarchy.

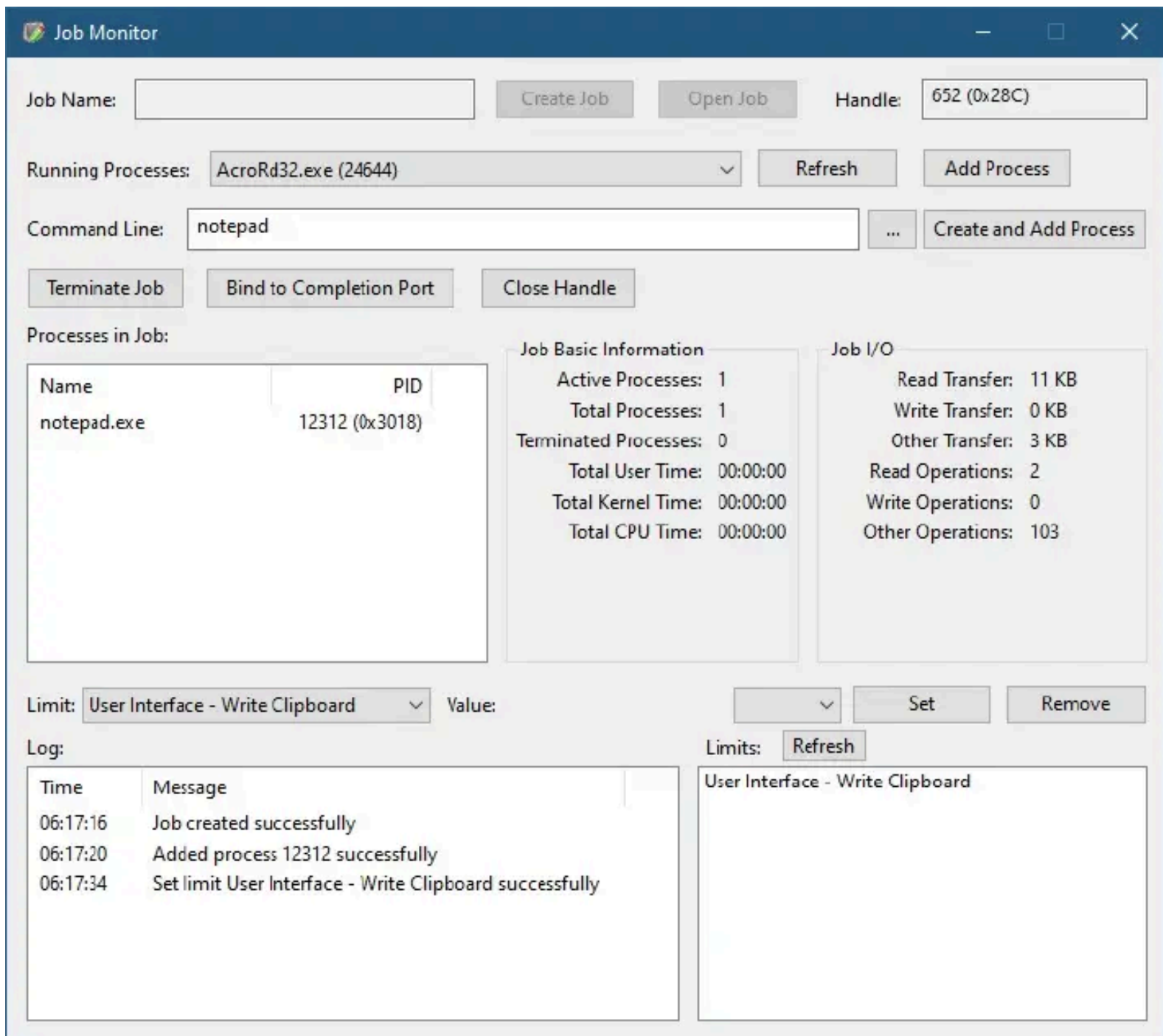
Table 4-6: User Interface job restrictions

UI flag (JOB_OBJECT_UILIMIT_*)	Description
NONE (0)	No limits
HANDLES (1)	Processes in the job cannot access USER handles (e.g. windows) owned by processes not part of the job
READCLIPBOARD (2)	Processes in the job cannot read data from the clipboard
WRITECLIPBOARD (4)	Processes in the job cannot write data to the clipboard
SYSTEMPARAMETERS (8)	Processes in the job cannot change system parameters by calling <code>SystemParametersInfo</code>
DISPLAYSETTINGS (0x10)	Processes in the job cannot call <code>ChangeDisplaySettings</code>
GLOBALATOMS (0x20)	Processes in the job cannot access global atoms. The job has its own atom table (see the next sidebar)
DESKTOP (0x40)	Processes in the job cannot create or switch desktops (<code>CreateDesktop</code> , <code>SwitchDesktop</code>)
EXITWINDOWS (0x80)	Processes in the job cannot call <code>ExitWindows</code> or <code>ExitWindowsEx</code>



An *Atom Table* is a system-managed table that maps strings (or integers in a certain range) to integers. Each entry in the table is an *atom*. These atoms are used with user interface APIs, such as when registering a window class (`RegisterClass` / `RegisterClassEx`) or by manually manipulating an atom table (`AddAtom`, `FindAtom`, `GlobalAddAtom` and others). The *Global atom table* is available to all applications - this is the one that is not accessible in a job restricted with `JOB_OBJECT_UILIMIT_GLOBALATOMS`.

Here is a quick experiment to show one effect of UI restrictions. Open *Job Monitor*, create a new job and insert a *Notepad* instance into it. Then set a UI limit of Write Clipboard (figure 4-21).

Figure 4-21: *Job Monitor* with a *Notepad* and UI restriction

Now open another *Notepad* instance outside of the job (or use any other text editing application). Copy some text from the application and try to paste it into the *Notepad* instance that is in the job. The operation should fail, even though the *Edit* menu shows the *Paste* option is enabled.

The `JOB_OBJECT_UILIMIT_HANDLES` flag prevents processes in the job from accessing other user interface objects (such as windows) outside the job. This means that calling functions such as `PostMessage` or `SendMessage` to windows outside the job fails. In some cases, there is a need to talk to a specific window outside the job from within the job. A process outside the job can grant (or remove) access to a window (or other USER object such as a menu or hook) by calling `UserHandleGrantAccess`:

```
BOOL UserHandleGrantAccess(  
    _In_ HANDLE hUserHandle,      // user object handle  
    _In_ HANDLE hJob,             // job handle  
    _In_ BOOL bGrant);           // TRUE to grant access, FALSE to remove it
```



The “hook” referred to in the previous paragraph is one of those that can be installed with `SetWindowsHookEx` (discussed in a later chapter). With this restriction in place, a process in the job cannot hook threads that run in processes outside the job.

Job Notifications

When job limits are violated, or when certain events occur, the job can notify an interested party via an I/O completion port, that can be associated with the job. I/O completion ports are typically used to handle completion of asynchronous I/O operations (which we’ll tackle in a later chapter), but in this special case are used as the mechanism of notifying when certain job events occur.

The job is a dispatcher (waitable) object, that becomes signaled when CPU time violation occurs. For this simple case, a thread can wait with `WaitForSingleObject` (as a common example) and then handle the CPU time violation. Setting a new CPU time limit resets the job to the non-signaled state.

The first step in getting notifications is to associate an I/O completion port with the job. Here is the relevant snippet from *JobMon* (`OnBindIoCompletion` function in *MainDlg.cpp*) (error handling omitted for clarity):

```

wil::unique_handle hCompletionPort(::CreateIoCompletionPort(
    INVALID_HANDLE_VALUE, nullptr, 0, 0));

JOB_OBJECT_ASSOCIATE_COMPLETION_PORT info;
info.CompletionKey = 0;      // application defined
info.CompletionPort = hCompletionPort.get();
::SetInformationJobObject(m_hJob.get(), JobObjectAssociateCompletionPortInforma\
tion,
    &info, sizeof(info));

// transfer ownership and store in a member
m_hCompletionPort = std::move(hCompletionPort);

```

Normally the first argument to `CreateIoCompletionPort` is a file handle, but in this case it's `INVALID_HANDLE_VALUE`, indicating no file is associated with the I/O completion port.

The next step is to wait for the completion port to fire by calling `GetQueuedCompletionStatus` defined like so:

```

BOOL GetQueuedCompletionStatus(
    _In_ HANDLE CompletionPort,
    _Out_ PDWORD pNumberOfBytesTransferred,
    _Out_ PULONG_PTR lpCompletionKey,
    _Out_ LPOVERLAPPED* pOverlapped,
    _In_ DWORD dwMilliseconds);

```

JobMon creates a thread and calls this function, waiting indefinitely until a notification arrives. A new thread is required in this case so that the UI thread of *JobMon* does not block, causing the UI to become unresponsive. Here is the relevant code from *JobMon* following the creation of the completion port:

```

// create a thread to monitor notifications
wil::unique_handle hThread(::CreateThread(nullptr, 0, [](auto p) {
    return static_cast<CMainDlg*>(p)->DoMonitorJob();
}, this, 0, nullptr));

```

Thread creation is explained in detail in the next chapter.

The thread is passed the `this` pointer so that it can conveniently call a member function (`DoMonitorJob`). `DoMonitorJob` calls `GetQueuedCompletionStatus` and responds when the wait is over:

```
DWORD CMainDlg::DoMonitorJob() {
    for (;;) {
        DWORD message;
        ULONG_PTR key;
        LPOVERLAPPED data;
        if (::GetQueuedCompletionStatus(m_hCompletionPort.get(),
            &message, &key, &data, INFINITE)) {
            // handle notification
        }
    }
}
```

The meaning of the parameters to `GetQueuedCompletionStatus` are special when used with job notifications (as opposed to a file). `pNumberOfBytesTransferred` is the notification type, summarized in table 4-7. The `CompletionKey` parameter is the same one specified in `CreateIoCompletionPort`, and is application-defined. Finally, `pOverlapped` is extra information, whose format depends on the type of notification (table 4-7).

Table 4-7: Job notifications

Notification (JOB_OBJECT_MSG_*)	Associated data (pOverlapped)	Description
END_OF_JOB_TIME (1)	NULL	Job time limit has been exhausted. The time limit is now canceled and processes in the job continue to run
END_OF_PROCESS_TIME (2)	The PID of the process	A process exceeded its per-process CPU time (the process is being terminated)
ACTIVE_PROCESS_LIMIT (3)	NULL	The active processes limit has been exceeded
ACTIVE_PROCESS_ZERO (4)	NULL	The number of active processes became zero (all processes exited for whatever reason)

Table 4-7: Job notifications

Notification (JOB_OBJECT_MSG_*)	Associated data (pOverlapped)	Description
NEW_PROCESS (6)	The PID of the new process	A new process was added to the job (either directly or because another process in the job created it). When the completion port is initially associated, all active processes are reported as well
EXIT_PROCESS (7)	The PID of the exiting process	A process in the job has exited
ABNORMAL_EXIT_PROCESS (8)	The PID of the exiting process	A process has exited abnormally, which means it terminated because of an unhandled exception, from a given list of exceptions (check the documentation for a complete list)
PROCESS_MEMORY_LIMIT (9)	The PID of the process	A process in the job has exceeded its memory consumption limit
JOB_MEMORY_LIMIT (10)	The PID of the process	A process in the job caused the job to exceed its job-wide memory limit
NOTIFICATION_LIMIT (11)	The PID of the process	(Windows 8+) A process in the job that registered a to receive a notification limit has exceeded a limit.

In case of a notification limit, call `QueryInformationJobObject` with `JobObjectNotificationLimitInformation` and/or `JobObjectNotificationLimitInformation2` (Windows 10) to query for the limits being violated.

The following code snippet from *JobMon* shows how some of these notification codes are handled:

```
switch (message) {
    case JOB_OBJECT_MSG_ACTIVE_PROCESS_LIMIT:
        AddLog(L"Job Notification: Active process limit exceeded");
        break;

    case JOB_OBJECT_MSG_ACTIVE_PROCESS_ZERO:
        AddLog(L"Job Notification: Active processes is zero");
        break;

    case JOB_OBJECT_MSG_NEW_PROCESS:
        AddLog(L"Job Notification: New process created (PID: "
            + std::to_wstring(PtrToUlong(data)) + L")");
        break;

    case JOB_OBJECT_MSG_EXIT_PROCESS:
        AddLog(L"Job Notification: process exited (PID: "
            + std::to_wstring(PtrToUlong(data)) + L")");
        break;

    case JOB_OBJECT_MSG_ABNORMAL_EXIT_PROCESS:
        AddLog(L"Job Notification: Process " + std::to_wstring(
            PtrToUlong(data)) + L" exited abnormally");
        break;

    case JOB_OBJECT_MSG_JOB_MEMORY_LIMIT:
        AddLog(L"Job Notification: Job memory limit exceed attempt by process "
            + std::to_wstring(PtrToUlong(data)));
        break;

    case JOB_OBJECT_MSG_PROCESS_MEMORY_LIMIT:
        AddLog(L"Job Notification: Process " + std::to_wstring(
            PtrToUlong(data)) + L" exceeded its memory limit");
        break;

    case JOB_OBJECT_MSG_END_OF_JOB_TIME:
        AddLog(L"Job time limit exceeded");
        break;

    case JOB_OBJECT_MSG_END_OF_PROCESS_TIME:
```

```

        AddLog(L"Process " + std::to_wstring(PtrToUlong(data))
              + L" has exceeded its time limit");
        break;
    }

```

AddLog is a private function that adds the corresponding message to the bottom list view.

For end-of-job time limit violation, the default action taken is to terminate all processes in the job. Each process' exit code is set to ERROR_NOT_ENOUGH_QUOTA (1816) and a notification is not sent. To change that, a call to SetInformationJobObject must be made beforehand to set a different end-of-job action with JobObjectEndOfJobTimeInformation information class, passing the following structure:

```

typedef struct _JOB_OBJECT_END_OF_JOB_TIME_INFORMATION {
    DWORD EndOfJobTimeAction;
} JOB_OBJECT_END_OF_JOB_TIME_INFORMATION, *PJOB_OBJECT_END_OF_JOB_TIME_INFORMATION;

#define JOB_OBJECT_TERMINATE_AT_END_OF_JOB 0
#define JOB_OBJECT_POST_AT_END_OF_JOB 1

```

A value of JOB_OBJECT_TERMINATE_AT_END_OF_JOB is the default, while JOB_OBJECT_POST_AT_END_OF_JOB causes posting a notification message without terminating the processes. If a completion port is not associated with the job, this value has no effect and the termination protocol is used.

Silos

Windows 10 version 1607 and Windows Server 2016 introduced an enhanced version of a job known as *Silo*. A silo always starts as a job, but it can be upgraded to a silo by using SetInformationJobObject with an undocumented information class, JobObjectCreateSilo (35), that appears in the Windows SDK headers, but is not documented. Some of the Silo APIs are documented in the *Windows Driver Kit* (WDK) for use by device driver writers. Since silos are mostly controllable from kernel mode, their programmatic usage is out of scope for this book.

There are two variations to silos: Application Silos and Server Silos. Server silos are only supported on Windows server machines, starting with Server 2016. They are used today

to implement *Windows Containers*, the ability to sandbox processes, creating a virtual environment that makes processes think they are on a machine of their own. This requires the redirection of file system, registry, and object namespace to be part of a particular silo, so the kernel had to go through significant changes internally to be silo-aware.

Application silos are used in applications that were converted to UWP using the *Desktop Bridge* technology. They are not nearly as powerful as server silos (nor they need to be). *Job Explorer* has a *Silo Type* column that indicates if a job is in fact a silo by listing its type. Figure 4-22 shows three application silos on the machine.



JobList	Name	Address	Active Process...	CPU Time	Total Process...	Silo ID	Silo Type	Child Jobs
0xFFFFC78D7C59000	\WorkerProcessSilo_0720BDE1-E83D-4A57-85D6-D0C8712...	0xFFFFC78D8C5C5650	0	00:00:03.031	2	844	App Silo	
0xFFFFC78D7FD18060	\Container_Microsoft.SkypeApp_14.54.910.x64_...tr8qe43...	0xFFFFC78DC52467D0	1	00:00:36.468	2	884	App Silo	
0xFFFFC78C8498F780	\Container_Microsoft.Windows.Common-UI_1.1916.5083.0_neutral_8...	0xFFFFC78DC8487D0	0	00:00:04.109	2	1340	App Silo	
0xFFFFC78D45FC60A0		0xFFFFC78D45FC60A0	1	00:00:01.375	1			1
0xFFFFC78D7DC78090		0xFFFFC78D7DC78090	1	00:00:15.656	1			1
0xFFFFC78D7E154060		0xFFFFC78D7E154060	1	00:00:00.109	1			1

Figure 4-22: *Job Explorer* showing Silos

Notice silos have a silo ID, which is a unique job ID that is used internally to identify silos.



More detailed information on silos can be found in the “Windows Internals, 7th edition, Part 1” in chapter 3.

Exercises

1. Write a tool called *MemLimit* that accepts a process ID and a number representing the maximum committed memory for the process and set that limit using a job.
2. Extend *JobMon* to cover all remaining limits that are not currently implemented, such as I/O and network limits.

Summary

Jobs offer many opportunities to control and limit processes, all implemented by the kernel itself. The introduction of nested jobs in Windows 8 makes jobs more useful and less restrictive.

In the next chapter, we'll start looking at threads. Processes and jobs are management objects, but threads are the actual ones assigned to processors to do work, so there is no OS life without threads.